PitchView — Relatório Técnico do Desenvolvimento

Projeto: PitchView, um software de análise de vídeos de futebol (Trabalho de Conclusão)

Repositório: `bomerich-prog/ANALISE-DE-FUTEBOL`

Branch oficial: `cursor/yolov8-video-analysis-5e9e` (PR #21 → `main`)

Período coberto: de 30/05/2026 (commit `6c8c3b4`) até 19/10/2026 (commit `bf0cc80`)

Sumário

- 0. Sobre este documento
- 1. Arquitetura geral
- 2. Estrutura dos principais arquivos
- 3. Diário técnico (ordem cronológica)
- 4. Estatísticas implementadas e como são calculadas
- 5. Funcionalidades pendentes e limitações conhecidas
- 6. Apêndice A — Linha do tempo de commits e PRs
- 7. Apêndice B — Testes automatizados
- 8. Apêndice C — Vídeos de teste usados

---

0. Sobre este documento

O relatório foi montado a partir de quatro fontes:

1. O histórico completo de conversas de desenvolvimento, com 82 pedidos e as respectivas respostas técnicas.
2. O `git log` da branch `cursor/yolov8-video-analysis-5e9e`, com as mensagens, os arquivos e as datas de cada commit.
3. As descrições dos Pull Requests #2 a #21.
4. O código atual da branch, de onde saíram as constantes, as funções e as classes citadas.

Algumas regras foram seguidas na escrita:

- Tudo o que está aqui foi implementado, medido ou discutido no projeto. Quando uma informação não existe, o texto diz isso explicitamente, por exemplo: "não há registro de discussão sobre este commit".
- Os commits de julho e agosto de 2026 (modelos Roboflow, ByteTrack, TeamClassifier, homografia, RF-DETR, busca híbrida e validação física) foram feitos em sessões cujo diálogo não está neste histórico. Essas seções descrevem só o que o commit, o PR e o código mostram. As subseções "Problemas encontrados" ficam curtas de propósito.
- Os números de tempo, frames e passes vêm das execuções relatadas em cada etapa. Execuções diferentes do mesmo vídeo podem dar resultados diferentes (ver Seção 3.33 e Seção 5).

Vídeos de referência (detalhes no Apêndice C):

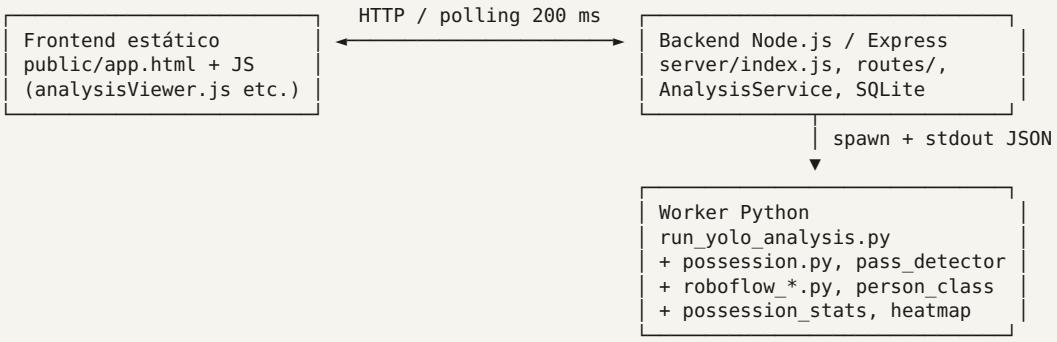
- **Clipe de 150 frames:** trecho de 6,1 s de Barcelona × Osasuna, 1920×1080. Foi o vídeo padrão até a etapa de recall da bola.
- **Gravação de tela "Gravação de Tela 2026-09-21 180218":** 401 frames, 30 fps, 1338×750, bola laranja. Foi o vídeo de testes de passes, posse e estatísticas a partir da Seção 3.27.

---

1. Arquitetura geral

1.1 Visão macro

O PitchView tem três camadas:



1. **Frontend.** O usuário envia o vídeo, inicia a análise e acompanha o modal "Visualização da Análise". O modal mostra o frame de debug ao vivo, a seção "Análise da Partida" (posse atual, passes por time, trocas de posse, bola), o cartão "Posse por Time" e o "Mapa de calor da bola".
2. **Backend Node.** `YoloAnalysisProvider.js` dispara o worker Python com `child_process.spawn`. Cada linha JSON do stdout vira `progress_stats`, guardado na memória (`progressStore`). O resultado final é gravado em `analyses.result_json`, no SQLite `data/app.db`.
3. **Worker Python.** Processa o vídeo quadro a quadro, grava o MP4 anotado (H.264), o `*.profile.json` de perfil e emite progresso.

1.2 Pipeline por quadro (ordem real no `analyze()` de `run_yolo_analysis.py`)

O diagrama pedido (Vídeo → YOLO → RF-DETR → Rastreamento → Times → Homografia → BallMemory → PossessionEngine → PassDetector → Estatísticas → Frontend) corresponde à ordem real abaixo. As detecções de pessoas e de bola correm em paralelo lógico no mesmo quadro, e a BallMemory fica dentro da etapa da bola.

```
Quadro do vídeo (OpenCV)
   ↓
[1] Detector de jogadores YOLO (football-player-detection.pt, imgsz 1280)
      classes: 0 ball | 1 goalkeeper | 2 player | 3 referee
      → a classe 0 (bola) é separada em player_ball_candidates (usada no fallback)
      → as classes humanas seguem
   ↓
[2] reject_non_person_detections (person_class.py): poste, bandeirinha, caixa fina/larga
   ↓
[3] ByteTrack (supervision): track_id por pessoa
   ↓
[4] Detector de keypoints do campo (football-pitch-detection.pt) → ViewTransformer (homografia)
   ↓
[5] Detecção da bola
      backend "yolo" (padrão): football-ball-detection.pt, imgsz 640
      backend "rfdetr": RF-DETR SoccerNet + ROI preditiva + Smart Crop / Semi-global / Global
   ↓
[6] NMS (IoU 0,1) → keep_recoverable_ball_candidates → seleção por ROI/confiança
   ↓
[7] apply_physical_ball_validation (anti-teleporte)
   ↓
[8] select_player_ball_fallback (classe ball do detector de jogadores, só continua a trilha)
   ↓
[9] BallTracker (roboflow_ball.py) + Ball Memory (5 quadros) + histórico de movimento
   ↓
[10] PersonClassLock.assign (person_class.py): Player / Referee / Goalkeeper travado por track
   ↓
[11] TeamClassifier (roboflow_team.py) + TeamTrackLock: time travado por track
   ↓
[12] resolve_goalkeepers_team_id: goleiro recebe o time do centróide mais próximo
   ↓
```

```
[13] assign_field_coordinates_to_detections: field_x / field_y em cm (campo 12000 × 7000)
   ↓
[14] BallHeatmapAccumulator.observe (ball_heatmap.py)
   ↓
[15] build_ball_state → PossessionEngine.update (possession.py)
   ↓
[16] PassDetector.update (pass_detector.py)
   ↓
[17] PossessionTeamAccumulator.observe(time, eventos de passe) (possession_stats.py)
   ↓
[18] Desenho do quadro anotado + emit(payload de progresso)
```

1.3 Papel de cada módulo

Módulo	Papel	Entrada	Saída
Detector de jogadores (YOLO Roboflow)	Encontrar pessoas e, como subproduto, bolas	Quadro BGR	Caixas de 4 classes com confiança
Filtro de não-pessoa	Tirar poste, bandeirinha e objetos finos antes do tracking	Caixas humanas	Caixas plausíveis
ByteTrack	Dar identidade (`track_id`) estável entre quadros	Caixas humanas	Caixas com `track_id`
Pitch keypoints + homografia	Projetar pixels no campo real	Quadro	`ViewTransformer` (≥ 4 keypoints)
Detector de bola (YOLO ou RF-DETR)	Encontrar a bola	Quadro, ou recorte/ tiles	Caixas de bola
Seleção, validação física e fallback	Escolher uma única bola plausível	Caixas de bola, previsão	Uma bola ou nenhuma
BallTracker + Ball Memory + predição	Continuidade da bola	Bola selecionada	Bola "observada", "lembrada" ou "prevista"
PersonClassLock	Impedir troca Player ↔ Referee ↔ Goalkeeper	Classe crua por track	Classe travada
TeamClassifier + TeamTrackLock	Time 0/1 estável por jogador	Recortes de jogadores	`team_id` travado
Goleiro → time	Atribuir time ao goleiro	Centróides dos times	`team_id` do goleiro
Coordenadas de campo	`field_x`, `field_y` em cm	Ponto dos pés / centro da bola	Coordenadas no campo
PossessionEngine	Quem tem a bola, com histerese	Jogadores + estado da bola	Snapshot de posse
PassDetector	Passes, turnovers, disputas, conduções, tabelas	Snapshots + registro de voo	Métricas e eventos
PossessionTeamAccumulator	Posse por time, com reatribuição de voo de passe	Time do possuidor + eventos	`possession_by_team`
BallHeatmapAccumulator	Grade 12 × 8 da bola observada	Bola com `field_x/ field_y`	`ball_heatmap`
Pontes Node (`passProgress.js` etc.)	Validar e copiar blocos do payload	Payload do worker	`progress_stats.*`
Frontend	Desenhar painel, barras e mapa	`progress_stats` / `result_json`	Interface

1.4 Comunicação worker → Node → frontend

```
run_yolo_analysis.py
   emit({"type":"progress", progress, message, framesProcessed, playersDetected,
         ballDetected, events, passes:{total_passes, events, turnovers},
         possession_by_team, ball_heatmap, playerDetections, goalkeeperDetections,
         refereeDetections, ballDetections, debugFrameJpeg?})
   ↓ (uma linha JSON no stdout)
YoloAnalysisProvider.js
   → passProgressFromPayload / possessionProgressFromPayload / ballHeatmapFromPayload
   → progressStore.set(analysisId, progress_stats)
   ↓
GET /api/analyses/:id         (progress_stats durante a análise; result_json no fim)
```

```
GET /api/analyses/:id/debug-frame  (JPEG em memória)
   ↓ polling de 200 ms (analysisClient.js)
analysisViewer.js → renderAnalysis → renderMatchEvents / renderPossessionByTeam / renderBallHeatmap
```

O payload é emitido quando o progresso muda 5 pontos percentuais, no intervalo do frame de debug ou quando a posse muda. Por isso passes, posse e heatmap aparecem em tempo real.

---

2. Estrutura dos principais arquivos

Os caminhos são relativos a `conversor-video/`.

2.1 Worker Python (`server/services/analysis/yolo/`)

Arquivo	Função
`run_yolo_analysis.py`	Orquestrador do pipeline. Lê o vídeo, carrega os modelos (`BallDetectorYOLO`, `BallDetectorRFDETR`), executa as etapas da Seção 1.2, desenha, emite progresso, grava o MP4 H.264 (`finalize_browser_video`), o perfil (`build_performance_profile_report`) e o diagnóstico de bola. Também contém as funções da bola: `select_ball_detection_with_roi`, `pick_detection_with_confidence_fusion`, `keep_recoverable_ball_candidates`, `select_player_ball_fallback`, `apply_physical_ball_validation`, `detect_ball_with_global_tiling`, `predict_motion_state_from_history`. Guarda ainda funções históricas do filtro de campo (`detect_field_region`, `apply_field_line_constraints`), que hoje não descartam jogadores.
`possession.py`	`PossessionEngine` (máquina de estados de posse) e `build_ball_state`. Define `TEAM_NAME_BY_ID = {0: "Team 0", 1: "Team 1"}`.
`pass_detector.py`	`PassDetector`: passes por placar, turnovers, disputas, conduções, tabelas, registro de voo, interceptação e velocidade física.
`possession_stats.py`	`PossessionTeamAccumulator`: frames por time, frames sem possuidor, reatribuição do intervalo de passe.
`ball_heatmap.py`	`BallHeatmapAccumulator`: grade 12 × 8 da bola observada.
`person_class.py`	`non_person_rejection_reason`, `reject_non_person_detections`, `PersonClassLock`.
`roboflow_team.py`	`TeamClassifier` (SigLIP + UMAP + KMeans), filtros de recorte de treino, âncora de cor e `TeamTrackLock`.
`roboflow_ball.py`	`BallTracker` (buffer 20) no estilo do repositório oficial da Roboflow.
`roboflow_view.py`	`ViewTransformer` (homografia com `cv2.findHomography`).
`roboflow_soccer_config.py`	Configuração do campo: 12000 × 7000 cm e vértices dos keypoints.
`compare_ball_detectors.py`	Script de comparação YOLO × RF-DETR para a bola.
`YoloAnalysisProvider.js`	Lança o worker, resolve o Python, lê o stdout, atualiza `progressStore`, guarda o frame de debug e monta o resultado.
`passProgress.js`	`passProgressFromPayload` → `{ total_passes, events, turnovers }`.
`possessionProgress.js`	Valida e copia `possession_by_team`.
`ballHeatmapProgress.js`	Valida e copia `ball_heatmap`.
`pythonResolver.js`	Escolhe o executável Python, ignorando o alias da Microsoft Store no Windows.
`models/`	Pesos `football-player-detection.pt`, `football-ball-detection.pt` e `football-pitch-detection.pt`, baixados por `scripts/download-pitch-model.js`.

2.2 Backend Node (`server/`)

Arquivo	Função
`server/index.js`	Servidor Express (`PORT` ou 3000).
`server/routes/analyses.js`	Rotas de análise. Inclui `GET /api/analyses/:id/debug-frame` (JPEG em memória) e streaming com `206 Partial Content`.
`server/services/analysis/AnalysisService.js`	Mapeia linhas do banco (`mapAnalysisRow`), sanitiza o resultado e recompõe `progress_stats` a partir do resultado final (`getProgressStatsFromResult`).
`server/services/analysis/AnalysisProvider.js` / `MockAnalysisProvider.js`	Contrato do provedor e versão mock com `events` placeholder.
`server/services/analysis/matchEvents.js`	`normalizePossessionTeam`, `normalizeMatchEvents`, `createPlaceholderMatchEvents` e `toUserFacingAnalysisMessage`.
`server/services/analysis/progressStore.js`	Guarda na memória o progresso e o último frame de debug.

2.3 Frontend (`public/`)

Arquivo	Função
`app.html`	Página da aplicação. Inclui o modal "Visualização da Análise", "Análise da Partida", "Posse por Time" e "Mapa de calor da bola".
`css/style.css`	Estilos do painel, das barras e do canvas.
`js/analysisClient.js`	Polling da análise (`POLL_INTERVAL_MS = 200`).
`js/analysisViewer.js`	`possessionLabel`, `backendPasses`, `readTurnovers`, `getEvents`, `renderMatchEvents`, `renderPossessionByTeam`, `renderBallHeatmap` e `renderAnalysis`.
`js/passCounts.js`	`countPassesByTeam`: conta eventos com `team` igual a `Team 0` ou `Team 1`.
`js/possessionCounts.js`	`readPossessionByTeam` e formatação de percentuais, frames e segundos.
`js/ballHeatmap.js`	`readBallHeatmap`, `cellIntensity` (raiz quadrada), `heatColor`, `hottestCell` e `drawBallHeatmap`.

2.4 Scripts e documentação

Arquivo	Função
`scripts/download-pitch-model.js`	Baixa os pesos Roboflow.
`scripts/test-*.py` / `test-*.mjs` / `test-*.js`	Testes automatizados (Apêndice B).
`docs/seminario-iii/`	Slides e roteiro do Seminário III.
`docs/relatorio-tecnico-desenvolvimento.md`	Este relatório.

3. Diário técnico (ordem cronológica)

Cada etapa segue a mesma estrutura: **1. Objetivo**, **2. Como funcionava antes**, **3. Qual era o problema**, **4. Solução adotada**, **5. Implementação**, **6. Fluxo do algoritmo**, **7. Problemas encontrados**, **8. Por que escolhemos essa solução**, **9. Impacto no restante do software** e **10. Resultado obtido**.

3.1 Ponto de partida: aplicação conversora de vídeo — `6c8c3b4` (30/05/2026)

1. Objetivo. Ter uma aplicação web onde o usuário se cadastra, envia vídeos de partidas, converte esses vídeos e mantém um histórico. Sobre essa base o projeto passou a ter visão computacional.

2. Como funcionava antes. Não havia código anterior. Este commit é a primeira versão do repositório ("Add files via upload").

3. Qual era o problema. A aplicação não analisava o conteúdo do vídeo. Já havia um esqueleto de pipeline: um `AnalysisProvider` mock, estados de análise e uma tela de histórico que iniciava a "análise" pela API.

4. Solução adotada. Manter essa estrutura (Node/Express + HTML/JS estático + SQLite) e encaixar a visão computacional como um novo *provider*, sem mudar o contrato das rotas.

5. Implementação. Estrutura existente em `conversor-video/`: - `server/index.js` com Express; - rotas em `server/routes/`; - serviços de análise em `server/services/analysis/` (`AnalysisService.js`, `AnalysisProvider.js`, `MockAnalysisProvider.js`); - banco SQLite em `data/app.db`; - frontend em `public/`.

6. Fluxo. Upload do vídeo ↓ conversão ↓ registro em `conversions` ↓ botão de análise ↓ `AnalysisService` cria a linha em `analyses` ↓ o provider mock devolve um resultado fictício.

7. Problemas encontrados. Na primeira leitura do repositório surgiu uma referência a `server.py`. Ao verificar a árvore real, ficou claro que a aplicação principal é Node/Express.

8. Por que essa base. O provider mock e os estados de pipeline já existiam. Integrar o YOLO como um provider novo preservava o resto do sistema.

9. Impacto. É a base de tudo o que vem depois.

10. Resultado. A aplicação web funcionava, com cadastro, upload, conversão e histórico.

3.2 Integração YOLOv8 e modal "Visualização da Análise" — `346e860`, `645a56f`, PR #2 (`7b54e66`) (05–08/06/2026)

1. Objetivo. Primeira fase de visão computacional. O pedido foi: - detectar jogadores e bola com YOLOv8 (Ultralytics); - desenhar as caixas com classe e confiança (ex.: "Player - 92%"); - criar o modal "Visualização da Análise" com vídeo processado, contador de jogadores, indicador de bola, frames processados e status.

A arquitetura deveria ficar pronta para as etapas futuras (YOLO → Tracking → Extração de Features → Machine Learning), sem implementar ainda tracking, features ou ML.

2. Como funcionava antes. A análise era um mock, sem nenhuma inferência.

3. Qual era o problema. - O vídeo anotado precisava tocar no navegador, então o codec tinha de ser compatível. - O streaming precisava ser autenticado. - O processamento pesado não podia bloquear o Node.

4. Solução adotada. - Um provider Node (`YoloAnalysisProvider.js`) lança um processo Python separado (`run_yolo_analysis.py`). - O worker escreve progresso em JSON no stdout. - Ao final, o worker produz um MP4 anotado e reencodado em H.264 (`645a56f`, "Encode YOLO output for browser playback"). - O vídeo é servido por rota autenticada com suporte a `206 Partial Content`.

5. Implementação. - `server/services/analysis/yolo/YoloAnalysisProvider.js` (novo). - `server/services/analysis/yolo/run_yolo_analysis.py` (novo). Usa `yolov8n.pt`, `--conf 0.25`, `--imgsz 640`, desenha caixas com `draw_detection`/`draw_label` e emite progresso com `emit()`. - `server/routes/analyses.js`: streaming do vídeo anotado. - `public/js/analysisViewer.js` (novo), `public/js/analysisClient.js`, `history.js`, `analyses.js`, `app.js`. - Modal em `public/app.html`. - `requirements.txt` com as dependências Python.

6. Fluxo. Usuário clica em analisar ↓ `AnalysisService` cria a análise ↓ `YoloAnalysisProvider` faz `spawn` do Python ↓ o worker lê cada quadro, roda YOLO, desenha as caixas e grava o quadro ↓ emite `{"type":"progress", ...}` ↓ o provider atualiza o status ↓ no fim, o MP4 é convertido para H.264 ↓ o frontend faz polling e mostra progresso e vídeo.

7. Problemas encontrados. As dependências de visão computacional tiveram de ser instaladas no ambiente de teste. Isso ficou registrado como recomendação de configuração ("Install `conversor-video/requirements.txt`...").

8. Por que essa solução. O processo separado isola a carga de CPU/GPU e deixa o worker trocável. Ele também segue a arquitetura modular pedida: cada etapa futura entra como bloco do worker.

9. Impacto. Criou o eixo provider ↔ worker ↔ frontend usado até hoje.

10. Resultado. Análise real com YOLOv8n, vídeo anotado no navegador, métricas e progresso. Testes feitos: - `node --check` em todos os JS alterados; - `py_compile` no worker; - execução real no vídeo de upload; - fluxo autenticado (registro → upload → análise → `206 Partial Content`); - teste manual gravado do modal.

3.3 Resolução do executável Python no Windows — `8262ac9`, PR #3 (06/06/2026)

1. Objetivo. Fazer o worker rodar no computador do usuário, que usa Windows.

2. Como funcionava antes. O provider chamava `python` diretamente.

3. Qual era o problema. No Windows, `python` pode apontar para o atalho da Microsoft Store (pasta `WindowsApps`). Esse atalho não executa o interpretador e quebra o `spawn`.

4. Solução adotada. Um resolvedor que testa candidatos em ordem e ignora o atalho da Store. A variável de ambiente `YOLO_PYTHON_BIN` tem prioridade.

5. Implementação. - `server/services/analysis/yolo/pythonResolver.js`: candidatos `py -3`, `py`, `python`, `python3`. Caminhos com `WindowsApps` são descartados. - Testes `scripts/test-python-resolver.js` e `scripts/test-yolo-spawn-error.js`, que cobrem o erro de spawn.

6. Fluxo. `YOLO_PYTHON_BIN` definido? Usa. Senão ↓ testa `py -3` ↓ `py` ↓ `python` ↓ `python3`, ignorando `WindowsApps` ↓ primeiro executável válido ↓ `spawn`.

7. Problemas encontrados. Não há outros registros além da descrição do PR.

8. Por que essa solução. É uma correção local no provider, sem impor configuração manual ao usuário.

9. Impacto. Só no lançamento do worker.

10. Resultado. O worker passou a iniciar no Windows.

3.4 Modo debug com frames ao vivo e correção da métrica de jogadores — `ab311ae` (08/06/2026)

1. Objetivo. - Ver, durante a análise, os quadros processados pelo YOLO com caixas e confiança, sem mudar o modo normal. - Investigar por que um vídeo com 17 jogadores e 1 árbitro resultou em "23 jogadores detectados".

2. Como funcionava antes. - O vídeo anotado só aparecia no fim. - O contador de jogadores somava detecções que podiam ser repetidas.

3. Qual era o problema. Detecções duplicadas (caixas muito sobrepostas sobre a mesma pessoa) inflavam a contagem. Era isso que levava 18 pessoas reais a aparecerem como 23.

4. Solução adotada. - **Debug opcional:** com `YOLO_DEBUG_LIVE_FRAMES=1`, o worker grava periodicamente um JPEG anotado (`live-frame.jpg`) que o frontend exibe. `YOLO_DEBUG_FRAME_INTERVAL` define a frequência. - **Métrica:** "jogadores detectados" passou a ser o **máximo por quadro**, depois de suprimir duplicatas por IoU (`suppress_duplicate_person_detections`, que usa `box_iou`). Foram criadas as métricas auditáveis `raw_player_detections` e `duplicate_player_detections_suppressed`.

5. Implementação. Arquivos: `run_yolo_analysis.py`, `YoloAnalysisProvider.js`, `AnalysisService.js`, `routes/analyses.js`, `analysisViewer.js`, `app.html`, `style.css`.

6. Fluxo. Quadro ↓ YOLO ↓ supressão de duplicatas por IoU ↓ contagem do quadro ↓ `max_players_detected = max(...)` ↓ se o debug estiver ativo e o quadro cair no intervalo, grava o JPEG ↓ o frontend exibe o último JPEG.

7. Problemas encontrados. O modo normal tinha de ficar exatamente igual. Sem debug, só o MP4 final aparece.

8. Por que essa solução. O máximo por quadro com deduplicação é a medida que corresponde a "quantas pessoas estão em campo". A soma bruta ou o último quadro não medem isso.

9. Impacto. Worker, provider, rotas e viewer. É a base da visualização em tempo real.

10. Resultado. Frames ao vivo funcionando com `ANALYSIS_PROVIDER=yolo YOLO_DEBUG_LIVE_FRAMES=1 YOLO_DEBUG_FRAME_INTERVAL=1 npm start`. A execução gerou `live-frame.jpg` com 150 atualizações. A métrica de jogadores deixou de contar duplicatas.

3.5 Filtro espacial do gramado por máscara verde (HSV) — `70dff1d`, PR #4 (`965de4a`), merge `2baefe8` (08–10/06/2026)

1. Objetivo. Considerar como jogadores só as pessoas dentro do gramado, descartando banco de reservas, comissão técnica, fotógrafos e pessoas na lateral. A meta dada como exemplo: 17 jogadores + 1 árbitro em campo e 5 pessoas fora resultariam em 18.

2. Como funcionava antes. Toda detecção `person` contava como jogador.

3. Qual era o problema. O contador de jogadores ficava errado quando havia pessoas fora do campo.

4. Solução adotada. - Detectar o gramado automaticamente por cor (máscara HSV verde). - Considerar o maior contorno como campo. - Descartar pessoas cujo **centro** da caixa caísse fora da

máscara. - Manter o árbitro na contagem se estivesse dentro. - No modo debug: contorno do campo em amarelo, detecções aceitas em verde, rejeitadas em vermelho.

5. Implementação. - Só `run_yolo_analysis.py`: `build_grass_support_mask`, `detect_field_region`, `is_detection_inside_field` e `draw_field_debug_overlay`. - Métricas novas: `raw_player_detections`, `filtered_player_detections` e `discarded_outside_field`.

6. Fluxo. Quadro ↓ máscara HSV do verde ↓ maior contorno ↓ YOLO ↓ para cada pessoa, o centro está dentro da máscara? sim: aceita; não: descarta ↓ métricas ↓ overlay de debug.

7. Problemas encontrados. - Turno 4: o merge com `main` gerou um conflito *add/add* simples em `run_yolo_analysis.py`. Ficou a versão da branch, mais nova. Merge `2baefe8`. - Mais tarde (Seção 3.8) ficou claro que placas de publicidade verdes também entravam na máscara.

8. Por que essa solução. Era a forma mais direta de separar "dentro" e "fora" sem um modelo novo, reaproveitando OpenCV.

9. Impacto. Só a etapa de filtragem de jogadores. Vídeo final e interface sem mudanças.

10. Resultado. No vídeo de teste: `players_detected=15`, `raw=2242`, `filtered=2242`, `discarded=0`. Nesse vídeo ninguém estava fora. Um teste sintético confirmou caixa dentro aceita e caixa fora rejeitada.

3.6 Robustez dos frames ao vivo no Windows e testes de sincronização — `29bd930` (PR #5), `f2acd3a`/`7d2f6f6` (PR #6), `64502d6` (PR #7), `849278a`/`9052472` (PR #8) (10/06–05/07/2026)

1. Objetivo. Fazer o frame ao vivo aparecer no computador Windows do usuário.

2. Como funcionava antes. O worker gravava `live-frame.jpg` com `os.replace` sobre um arquivo temporário.

3. Qual era o problema. - No Windows, `os.replace` falhava com `WinError 5` (acesso negado) quando o arquivo estava aberto pelo servidor. - O usuário também não via os frames ao vivo e desconfiou da sincronização do Git.

4. Solução adotada. - **PR #5:** repetir `os.replace` quando ocorre `WinError 5`. - **PR #6:** ligar os frames ao vivo por padrão. - **PR #7:** log temporário de diagnóstico do live frame no Windows. - **PR #8:** um teste visual para provar que commits chegavam ao computador do usuário. O botão "Selecionar arquivos" ficou vermelho, só no CSS.

5. Implementação. - `run_yolo_analysis.py`: retry em `write_live_debug_frame` e constante de padrão do live frame. - `public/css/style.css` (PR #8). - Merges `29eedd5`, `bf31cee` e `95e4c7e` levaram o retry para outras linhas.

6. Fluxo. Gravar JPEG temporário ↓ `os.replace` ↓ se `WinError 5`, aguardar e tentar de novo ↓ servidor serve o arquivo.

7. Problemas encontrados. O teste do botão vermelho confirmou que commit, push e pull funcionavam. O problema estava só na visualização em tempo real, o que levou à Seção 3.7.

8. Por que essa solução. Um retry curto é a correção típica para o bloqueio de arquivo do Windows.

9. Impacto. Só no caminho do frame de debug.

10. Resultado. O usuário continuou sem ver os frames. Isso motivou a troca do mecanismo por frames em memória.

3.7 Visualização em tempo real com frames em memória — `ca626aa` (05/07/2026)

1. Objetivo. Ter uma visualização ao vivo robusta das detecções durante a análise. Não era obrigatório manter o `live-frame.jpg`.

2. Como funcionava antes. O frame era um arquivo em disco, lido pelo servidor.

3. Qual era o problema. Disputa de arquivo no Windows e dependência do sistema de arquivos.

4. Solução adotada. - O worker codifica o quadro anotado em JPEG e envia como **base64 dentro da linha de progresso** (`debugFrameJpeg`). - O Node guarda o último frame na memória (`progressStore`). - A rota `GET /api/analyses/:id/debug-frame` serve esse buffer como `image/jpeg`. - O polling do frontend caiu para 200 ms.

5. Implementação. - `run_yolo_analysis.py`: `encode_live_debug_frame`; `--debug-frame-dir memory`. - `server/services/analysis/progressStore.js`. - `YoloAnalysisProvider.js`: converte `debugFrameJpeg` em `debug_frame_url` e `debug_frame.data`. - `AnalysisService.js`, `routes/analyses.js`. - `public/js/analysisClient.js`: `POLL_INTERVAL_MS = 200`.

6. Fluxo. Quadro anotado ↓ `cv2.imencode('.jpg')` ↓ base64 ↓ `emit({... debugFrameJpeg})` ↓ provider guarda em memória e publica `progress_stats.debug_frame_url` ↓ frontend faz polling de `/api/analyses/:id` ↓ se houver `debug_frame_url`, busca `/debug-frame` ↓ exibe o JPEG.

7. Problemas encontrados. No turno 6, o usuário ainda via "O vídeo processado aparecerá quando a análise terminar". O diagnóstico, sem alterar código, encontrou a primeira diferença entre os ambientes: o teste do agente iniciou o servidor com `YOLO_DEBUG_LIVE_FRAMES=1`, e `npm start` sozinho executa só `node server/index.js`. Itens verificados: - **Porta:** 3005 (agente) contra 3000 (usuário) não era a causa. - **Rota esperada:** `/debug-frame` deve responder `200 image/jpeg`. - **Persistência:** frames em memória não sobrevivem a um reinício do servidor.

8. Por que essa solução. Elimina o arquivo intermediário e a corrida de escrita no Windows. Reaproveita o canal stdout que já existia.

9. Impacto. Provider, rotas, polling e viewer. O MP4 final não mudou.

10. Resultado. Visualização em tempo real funcionando quando o servidor sobe com a variável de debug, validada em vídeo longo antes da conclusão.

3.8 Polígono do campo pelas linhas brancas (três versões) — `10c5490`, `4122de8`, `a4c6150` (06/07/2026)

1. Objetivo. Substituir a máscara verde (HSV) por uma área jogável mais confiável, usando as **linhas brancas** do campo como referência principal. Manter só os jogadores cujos **pés** (ponto inferior central da caixa) caem dentro do polígono, e desenhar o polígono no debug e no vídeo final.

2. Como funcionava antes. Máscara verde e centro da caixa (Seção 3.5).

3. Qual era o problema. Placas de publicidade verdes eram consideradas gramado. Pessoas fora da linha lateral eram aceitas como jogadores.

4. Solução adotada (evolução). - **Versão 1 (`10c5490`):** - detectar linhas brancas com OpenCV (`build_white_line_mask`, `detect_field_line_segments`); - usar as linhas para escolher o contorno verde correto (`select_grass_contour_by_field_lines`); - filtrar pelos pés (`detection_foot_point`); - desenhar o polígono. - **Versão 2 (`4122de8`):** turno 9. A imagem enviada pelo usuário mostrou que o polígono continuava seguindo a publicidade. A causa: as linhas só **selecionavam** o contorno e não **recortavam** o polígono. A correção transforma linhas longas próximas da borda em **restrições de**

meio-plano (`find_field_boundary_constraints`, `clip_polygon_with_constraint`, `apply_field_line_constraints`). Assim, a publicidade conectada à lateral não expande mais a área. - **Versão 3 (`a4c6150`):** turno 10. Só as linhas de delimitação (laterais e de fundo) podem limitar o polígono. Travessão, postes, rede e marcações internas ficam de fora. `filter_external_boundary_constraints`: - agrupa linhas paralelas pelo lado do campo; - mantém a mais externa em relação ao centro; - descarta verticais internas no centro da imagem.

5. Implementação. Só `run_yolo_analysis.py`: funções `build_white_line_mask`, `detect_field_line_segments`, `build_segment_mask`, `sample_grass_ratio_along_line_side`, `find_field_boundary_constraints`, `filter_external_boundary_constraints`, `clip_polygon_with_constraint`, `apply_field_line_constraints`, `select_grass_contour_by_field_lines` e `detect_field_region`.

6. Fluxo (versão final). Quadro ↓ máscara verde de apoio ↓ máscara de linhas brancas ↓ segmentos de linha ↓ escolha do contorno verde coerente com as linhas ↓ fecho convexo ↓ classificação das linhas (só as externas) ↓ recorte do polígono por meio-plano em cada linha externa ↓ para cada pessoa, os pés estão no polígono? ↓ aceita ou descarta ↓ desenha o polígono.

7. Problemas encontrados. - A versão 1 não resolveu o caso real porque a máscara verde ainda dominava. - A versão 2 resolveu a publicidade, mas usava qualquer linha longa, inclusive marcações internas. - A versão 3 restringiu às linhas externas.

8. Por que essa solução. As linhas laterais e de fundo são o limite geométrico real do campo. A cor verde não distingue gramado de placa.

9. Impacto. Só a filtragem de jogadores e o desenho. Mais tarde o filtro de rejeição humana foi removido (Seção 3.13).

10. Resultado. Testes sintéticos (publicidade verde conectada à lateral) passaram: a pessoa na placa foi rejeitada e a pessoa no campo, aceita. O polígono foi validado no vídeo real.

3.9 Modelo de keypoints do campo — `81792a2`, `e6d9ba4` (PR #9), merge `95e4c7e` (27/07/2026)

> Esta etapa e as Seções 3.10 a 3.15 foram feitas em sessões cujo diálogo não está neste histórico. O conteúdo vem dos commits, dos PRs e do código.

1. Objetivo. Detectar os pontos de referência do campo (keypoints) em cada quadro. É o pré-requisito da homografia.

2. Como funcionava antes. Só havia o polígono por linhas brancas, sem pontos de referência métricos.

3. Qual era o problema. Sem keypoints não há como projetar posições de imagem para coordenadas reais do campo.

4. Solução adotada. - Carregar o modelo `football-pitch-detection.pt` (Roboflow) ao lado do detector YOLO. - Inferir os keypoints em todos os quadros processados. - Desenhá-los no vídeo anotado e no frame ao vivo. - Download automático do peso.

5. Implementação. - `scripts/download-pitch-model.js` (novo) e `package.json` (`81792a2`). - `run_yolo_analysis.py`: argumentos `--pitch-model` e `--pitch-keypoint-conf`; funções `extract_pitch_keypoints`, `draw_pitch_keypoints` e `resolve_pitch_model_path`. - O merge da branch de retry do WinError (`95e4c7e`) preservou o retry de `os.replace` e o `debugFrameJpeg`.

6. Fluxo. Quadro ↓ `pitch_model(frame)` ↓ keypoints acima da confiança ↓ desenho.

7. Problemas encontrados. Conflitos em `run_yolo_analysis.py` com a branch do WinError, resolvidos mantendo as duas funcionalidades (descrição do PR #9).

8. Por que essa solução. Modelo pronto do ecossistema Roboflow Sports, treinado para campos de futebol.

9. Impacto. Mais uma inferência por quadro. Na Seção 3.21 a etapa "pitch" respondia por 26,3% do tempo.

10. Resultado. Keypoints desenhados em todos os quadros. A base para a homografia ficou pronta (Seção 3.13).

3.10 Modelo dedicado de bola, BallTracker e InferenceSlicer — `e6111f2`, `401dcd4`, `c794580`, `0bd7d31`, `8d12676`, `2c92ba9` (27/07–01/08/2026)

1. Objetivo. Melhorar a detecção da bola com um modelo especializado e um rastreador de bola.

2. Como funcionava antes. A bola vinha do YOLOv8 genérico (`sports ball`).

3. Qual era o problema. O modelo genérico não é treinado para bola de futebol pequena em transmissão.

4. Solução adotada (evolução). - `e6111f2`: modelo dedicado `football-ball-detection.pt`, com **fallback** para o detector base quando o dedicado não achava nada. - `401dcd4`: o fallback para o detector base foi **desativado temporariamente**. - `c794580`: `BallTracker` no estilo Roboflow (`roboflow_ball.py`). Mantém um buffer de posições recentes e, entre várias caixas, escolhe a mais próxima do centro recente. - `0bd7d31` + `8d12676`: inferência da bola fatiada com `sv.InferenceSlicer`, alinhada ao fluxo oficial, com os argumentos ajustados ao supervision 0.29.1. - `2c92ba9`: o caminho do `InferenceSlicer` foi **desativado temporariamente**. O código atual mantém o callback (`ball_slicer_callback`) e `ball_slicer = None`, com o comentário de que a ligação foi preservada para reativação.

5. Implementação. - `run_yolo_analysis.py`: `BallDetectorYOLO` e `resolve_ball_model_path`. - `roboflow_ball.py`: `BallTracker`, `BALL_TRACKER_BUFFER_SIZE = 20`. - `requirements.txt`. - `download-pitch-model.js` passou a baixar também o modelo de bola.

6. Fluxo. Quadro ↓ `football-ball-detection.pt` (imgsz 640) ↓ caixas ↓ `BallTracker.update` ↓ uma caixa.

7. Problemas encontrados. O fallback e o slicer foram desligados logo depois de entrarem. Não há registro do motivo nesta conversa.

8. Por que essa solução. Modelo especializado mais o rastreador oficial, sem uma segunda rede por padrão.

9. Impacto. Etapa da bola. O `BallTracker` continua no pipeline atual.

10. Resultado. Bola com modelo dedicado e rastreador. O slicer ficou disponível no código, mas desligado.

3.11 Detector oficial de jogadores (Roboflow) e ByteTrack — `aafb038` (PR #11), `72f8ba8`, `97700b5` (31/07–03/08/2026)

1. Objetivo. Separar jogador, goleiro, árbitro e bola com um modelo especializado, e dar identidade estável a cada pessoa ao longo do vídeo.

2. Como funcionava antes. YOLOv8n genérico, só com a classe `person`, sem identidade entre quadros.

3. Qual era o problema. O modelo genérico não separa árbitro de jogador. Sem tracking, não há como falar de "o mesmo jogador" em quadros diferentes.

4. Solução adotada. - `aafb038`: `football-player-detection.pt` por padrão, mapa de classes `ball=0`, `goalkeeper=1`, `player=2`, `referee=3`, com as quatro classes desenhadas separadamente. - `72f8ba8`: `sv.ByteTrack` para as detecções humanas. - `97700b5`: ajuste de persistência do ByteTrack e suavização Player/Referee por maioria em 9 quadros (`stabilize_player_referee_class`, `PLAYER_REFEREE_HISTORY_WINDOW = 9`).

5. Implementação. - `run_yolo_analysis.py`: `resolve_player_model_path`; criação do `ByteTrack` com `minimum_consecutive_frames=3` (`BYTE_TRACK_MIN_CONSECUTIVE_FRAMES`), `lost_track_buffer=60` e `minimum_matching_threshold=0.72`. - `YoloAnalysisProvider.js`, `download-pitch-model.js`, `package.json`.

6. Fluxo. Quadro ↓ detector de jogadores (imgsz 1280) ↓ separa classes ↓ ByteTrack nas pessoas ↓ histórico de classe por track ↓ maioria em 9 quadros.

7. Problemas encontrados. Na época, a classe `ball` do detector de jogadores era **descartada**. Na Seção 3.27 descobrimos que ela enxergava a bola laranja melhor que o modelo dedicado.

8. Por que essa solução. Modelo e tracker do mesmo ecossistema (Roboflow Sports + supervision).

9. Impacto. É a base de times, posse e passes. A detecção de jogadores passou a ser a inferência mais cara (Seção 3.21: 26,1%).

10. Resultado. Quatro classes e `track_id` por pessoa.

3.12 TeamClassifier v1 e atribuição de goleiros — `a329aaa`, `1abca2f`, `e978a5b`, `f7aec46`, `4bd2549`, `e4cfccb` (01–08/08/2026)

1. Objetivo. Classificar automaticamente cada jogador em um de dois times, sem rótulos, e atribuir o goleiro ao time certo.

2. Como funcionava antes. Não havia noção de time.

3. Qual era o problema. As cores dos uniformes variam de jogo para jogo. A classificação tem de ser não supervisionada.

4. Solução adotada. - `a329aaa`: `TeamClassifier` oficial (`roboflow_team.py`). Os recortes passam pelo SigLIP (`google/siglip-base-patch16-224`, média do *hidden state*), depois UMAP para 3 dimensões, depois KMeans com k = 2. - `1abca2f` + `e978a5b`: qualidade dos recortes de treino (`classify_training_crop_quality`: vazio, pequeno, na borda; `is_crop_blurry` com variância do Laplaciano ≥ 45). - `f7aec46`: **fit único e limitado** antes da análise. A cada 10 quadros (`TEAM_CLASSIFIER_STRIDE`), até 400 quadros, coleta até 60 recortes (`TARGET_SAMPLES`) com altura ≥ 50 e largura ≥ 30. - `4bd2549`: heurística oficial do goleiro (`resolve_goalkeepers_team_id`). O goleiro recebe o time cujo centróide de pés de jogadores está mais próximo. - `e4cfccb`: cache da predição por `track_id`, com refresh a cada 30 quadros (`TEAM_CLASSIFIER_REFRESH_INTERVAL`).

5. Implementação. - `roboflow_team.py` (novo). - `run_yolo_analysis.py`: coleta de recortes, `get_player_crops_from_payload`, `player_team_cache_by_track_id`. - `requirements.txt` com SigLIP/ transformers, UMAP e scikit-learn.

6. Fluxo. Pré-passagem: lê o vídeo, a cada 10 quadros pega recortes de `Player` válidos ↓ 60 recortes ↓ `fit` ↓ análise: para cada jogador com track, há cache com menos de 30 quadros? usa o cache; senão, `predict` e grava ↓ goleiros: centróide mais próximo.

7. Problemas encontrados. Os problemas desta versão só foram diagnosticados depois (Seções 3.21 e 3.22): - sem `random_state`; - refresh que sobrescrevia o time; - recortes de árbitro e bandeira no fit; - time 0 usado como fallback.

8. Por que essa solução. É a solução oficial do Roboflow Sports, sem treino supervisionado.

9. Impacto. Criou o `team_id` usado pela posse e pelos passes.

10. Resultado. Times atribuídos automaticamente, com cache por track. A instabilidade de rótulo continuou até `1fb0cef`.

3.13 Homografia oficial, perfil de desempenho e remoção do filtro de campo — `30a400b`, `96c87c5`, `d761c60`, `4db3f11` (03–09/08/2026)

1. Objetivo. Converter posições da imagem para **centímetros no campo**, medir o custo de cada etapa e retirar um filtro que prejudicava a detecção.

2. Como funcionava antes. - Posições só em pixels. - Nenhum perfil de tempo por etapa. - Filtro de campo rejeitando pessoas.

3. Qual era o problema. - Pixels variam com zoom e enquadramento. - O filtro de máscara de campo descartava jogadores reais perto das placas de LED. Esse motivo foi citado no diagnóstico da Seção 3.21: "não o filtro antigo que comia jogadores no LED".

4. Solução adotada. - `30a400b`: homografia oficial. - `roboflow_soccer_config.py` define o campo de **12000 × 7000 cm** e os vértices. - `roboflow_view.py` implementa o `ViewTransformer` com `cv2.findHomography`. - A homografia só é montada com pelo menos 4 keypoints válidos (`HOMOGRAPHY_MIN_KEYPOINTS = 4`). - Os pés dos jogadores e o centro da bola ganham `field_x`/`field_y`. - `96c87c5`: perfil detalhado (`*.profile.json`) com tempo por etapa e hardware. - `d761c60`: o filtro de rejeição humana pela máscara do campo foi removido. - `4db3f11`: a sobreposição visual da máscara foi removida.

5. Implementação. - `run_yolo_analysis.py`: `build_view_transformer_from_pitch_keypoints`, `assign_field_coordinates_to_detections`, `transform_image_point_to_field`, `transform_field_point_to_image`, `create_profile_state`, `record_profile_stage`, `build_performance_profile_report`, `write_performance_profile_report`. - Arquivos novos: `roboflow_soccer_config.py` e `roboflow_view.py`.

6. Fluxo. Keypoints ↓ ≥ 4 válidos? ↓ `ViewTransformer(source=pixels, target=vértices do campo)` ↓ para cada pessoa: pés → `field_x/field_y`; bola: centro → `field_x/field_y` ↓ o perfil registra o tempo de cada etapa.

7. Problemas encontrados. Depois da remoção, o filtro espacial ficou como código morto: `accepted_player_detections = list(player_detections)` (constatado na Seção 3.21). Bandeirinha e objetos finos passaram a entrar como jogador até a Seção 3.24.

8. Por que essa solução. Coordenadas métricas permitem raios de posse e distâncias de passe independentes da câmera. O perfil permite atacar o desempenho com dados.

9. Impacto. `field_x/field_y` passaram a ser a base da posse (Seção 3.19), dos passes, da posse por time e do mapa de calor.

10. Resultado. Coordenadas de campo em todos os quadros com homografia válida. O perfil por etapa ficou disponível para todas as execuções seguintes.

3.14 Ball Memory, predição e fusão por ROI — `80128b3`, `5a11878`, PR #10 (`4cecdc6`), merge `12e6e73`, diagnóstico `399b73d` (11–12/08/2026)

1. Objetivo. Reduzir as interrupções da trilha da bola, mantendo uma única inferência de bola por quadro.

2. Como funcionava antes. Detecção quadro a quadro com `BallTracker`. Um quadro sem caixa interrompia a trilha.

3. Qual era o problema. A bola some por poucos quadros (oclusão, borrão), e a trilha quebrava.

4. Solução adotada (descrição do PR #10). - **Ball Memory:** até 5 quadros sem detecção (`BALL_MEMORY_MAX_MISSING_FRAMES = 5`). A continuidade é só interna: nenhuma caixa falsa é desenhada. - **Predição de movimento** a partir dos centros recentes (`predict_ball_center`, histórico `BALL_TRAJECTORY_HISTORY_SIZE = 6`). - **Seleção por ROI primeiro:** procurar perto da posição prevista e, se não houver nada, no quadro todo (`select_ball_detection_with_roi`). O raio da ROI é o maior entre 45 px (`BALL_ROI_MIN_RADIUS_PX`) e 2,25 × o tamanho da caixa anterior (`BALL_ROI_SCALE_FROM_BOX`). - **Fusão de confiança:** limiar menor perto da previsão (0,12 até 80 px) e mais exigente longe (0,35) (`pick_detection_with_confidence_fusion`, `ball_confidence_threshold_for_distance`). - **Recuperação adaptativa:** depois de mais de 5 falhas seguidas, zera previsão e memória. - `5a11878`: a contagem de continuidade passou a considerar o estado interno da memória. - `399b73d`: log de diagnóstico da bola por quadro (`build_ball_diagnostics_summary`, histogramas de confiança).

5. Implementação. Só `run_yolo_analysis.py`. Métricas do perfil em `ball_tracking_enhancements`: `ball_memory_hits`, `ball_memory_misses`, `ball_roi_searches`, `ball_roi_success`, `ball_prediction_error`, `ball_track_interruptions_before/after` e `ball_track_continuity_gain_percent`.

6. Fluxo. Detecção da bola ↓ há previsão? ↓ procura na ROI ao redor dela ↓ candidata perto (≤ 80 px) com confiança ≥ 0,12, ou longe com ≥ 0,35? ↓ aceita; senão, busca global ↓ nenhuma? memória mantém a trilha (≤ 5 quadros) ↓ mais de 5? zera histórico e previsão.

7. Problemas encontrados. - O merge `12e6e73` resolveu um conflito em `run_yolo_analysis.py` com `main`. - Mais tarde (Seção 3.25) foi notado que a memória **não desenha** a última bola e apaga o histórico no sexto quadro, o que prolonga buracos.

8. Por que essa solução. Continuidade sem segunda rede, sem aumentar a resolução e sem caixas sintéticas.

9. Impacto. O estado observado/lembrado/previsto passou a existir e é usado pela posse (`build_ball_state`).

10. Resultado (mesmo clipe):

Métrica	Valor
Interrupções	11 → 3 (ganho de continuidade de 72,73%)
Recuperações pela memória	35
Erro médio de previsão	56,8 px

Métrica	Valor
Buscas por ROI	16, com 15 sucessos
Tempo total	398,32 s → 387,71 s (−2,67%)

3.15 RF-DETR SoccerNet, ROI preditiva, busca híbrida, semi-global e validação física — `7960aaf`, `26671f4`, `09e2bac`, `804682c`, `5cfb5c4`, `f71f1dc`, `c67eace` (12–25/08/2026)

1. Objetivo. Oferecer um detector de bola mais robusto (RF-DETR SoccerNet) e estratégias de busca para não perder bolas pequenas e rápidas, sem aceitar saltos impossíveis.

2. Como funcionava antes. Só YOLO de bola em quadro inteiro, com memória e ROI (Seção 3.14).

3. Qual era o problema. Bolas pequenas ou rápidas se perdiam. Ao procurar com limiares mais baixos, surgia o risco de "teleporte": aceitar uma caixa falsa longe da trilha.

4. Solução adotada (evolução). - **`7960aaf` / `26671f4`, RF-DETR comutável:** - `BallDetectorRFDETR` carrega o checkpoint `weights/checkpoint_best_regular.pth` do repositório `julianzu9612/RFDETR-Soccernet` (Hugging Face), com download por `ensure_rfdetr_soccernet_files`. - `--ball-detector yolo|rfdetr` escolhe o backend. O padrão é `BALL_DETECTOR_BACKEND_DEFAULT = "yolo"`. - `compare_ball_detectors.py` compara os dois. - `26671f4` corrigiu o carregamento do checkpoint. - **`09e2bac`, rastreamento preditivo com ROI adaptativa (só RF-DETR):** - a ROI cresce 200 → 300 → 450 → 650 px a cada falha (`PREDICTIVE_ROI_SIZE_SEQUENCE`); - depois de 4 falhas, volta para o quadro todo (`PREDICTIVE_GLOBAL_FALLBACK_AFTER_MISSES`); - limiares 0,08 perto e 0,20 longe; - o estado de movimento (`vx`, `vy`) vem de `predict_motion_state_from_history`. - **`804682c`, busca híbrida:** - *Smart Crop*: recorte adaptativo de 250 → 500 → 900 → 1400 px (`CROP_EXPANSION_LEVELS`, `build_adaptive_crop_schedule`, `pick_adaptive_crop_size`); - *Global tiling*: tiles de 640 px com 25% de sobreposição (`detect_ball_with_global_tiling`, `build_tile_ranges`); - limiares 0,05 perto e 0,1 longe, com bônus de proximidade. - **`5cfb5c4`, estágio semi-global:** antes do tiling do quadro todo, faz tiling em uma janela de 1200 px ao redor da previsão (`SEMI_GLOBAL_WINDOW_SIZE`). Só se ela vier vazia depois do NMS cai no *full_global*. - **`f71f1dc`, recuperação semi-global precoce:** depois de um sucesso semi-global, o crop começa no tamanho máximo e encolhe ao longo de 10 quadros (`SEMI_GLOBAL_RECOVERY_COOLDOWN_FRAMES`). - **`c67eace`, validação física** (`apply_physical_ball_validation`, ligada por padrão com `PHYSICAL_BALL_VALIDATION=on`): - até 120 px da previsão: aceita; - de 120 a 300 px: fica **pendente** e precisa de 2 confirmações seguidas a menos de 120 px da pendente; - acima de 300 px: rejeita e conta como "teleporte evitado"; - o estado é resetado (aceita sem condição) se não houver previsão, se não houver última detecção, depois de mais de 10 quadros sem detecção ou com velocidade ≥ 120.

5. Implementação. - `run_yolo_analysis.py`: `BallDetectorRFDETR`, `build_ball_detector`, `load_rfdetr_ball_class_id`, `download_huggingface_model_file`, `build_square_roi_from_center`, `translate_detections_to_full_frame`, `merge_detection_batches`, `compute_nms_keep_mask`, `apply_physical_ball_validation`. - `compare_ball_detectors.py`. - `requirements.txt` com `rfdetr` e `huggingface_hub`.

6. Fluxo (backend RF-DETR, com busca híbrida ligada). Estado `smart_crop`: recorte do tamanho do cronograma ao redor da previsão ↓ RF-DETR no recorte ↓ ≥ 3 quadros sem bola? muda para `global_search` ↓ `global_search`: tiles em janela de 1200 px (`semi_global`) ↓ vazio depois do NMS? tiles no quadro todo (`full_global`) ↓ 3 detecções seguidas? volta para `smart_crop` ↓ NMS (IoU 0,1) ↓ seleção com fusão de confiança ↓ validação física ↓ BallTracker.

Com o backend `yolo` (padrão), a predição só alimenta a seleção e a validação física. ROI preditiva, crop e tiling **não rodam**.

7. Problemas encontrados. - Os pesos do RF-DETR não estavam disponíveis nos ambientes de teste posteriores. Por isso a comparação de recall na Seção 3.27 não pôde incluir esse backend. - O diagnóstico da Seção 3.21 constatou que, com o padrão `yolo`, toda a busca inteligente ficava desligada.

8. Por que essa solução. Combinar busca local barata (crop) com busca ampla só quando necessária (semi-global, depois global) reduz o custo de procurar a bola em todo o quadro. A validação física impede que limiares mais baixos criem saltos impossíveis.

9. Impacto. Etapa da bola. A validação física roda em qualquer backend e afeta posse e passes, porque segura saltos grandes por 2 quadros.

10. Resultado. Pipeline de bola com dois backends e busca em três níveis. Essas melhorias foram apresentadas no Seminário III (Seção 3.16).

3.16 Seminário III — `169fcb7`, `0a53870`, `d98e913` (10/09/2026)

1. Objetivo. Preparar a apresentação de andamento do TCC, mostrando avanço em relação ao Seminário II.

2. Como funcionava antes. Só existia o material do Seminário II, fornecido pelo usuário.

3. Qual era o problema. Era preciso mostrar a evolução técnica de forma clara para a banca.

4. Solução adotada (três versões). - `169fcb7`: deck com 7 slides e capturas reais do sistema rodando com YOLO em tempo real: capa, ponto de partida, sistema funcional, YOLO ao vivo, frame com caixas, filtragem espacial e próximos passos. - `0a53870`: deck só com texto, com o conteúdo ditado pelo usuário em 5 slides: - evolução do projeto; - detecção da bola: RF-DETR SoccerNet, Predictive Ball Tracking, Smart Crop, Semi Global Search, Ball Memory; - rastreamento inteligente; - evolução do pipeline: homografia, coordenadas, times, goleiros, cache, remoção da máscara visual; - estado atual. - `d98e913`: os próximos passos do último slide passaram a ser: - melhorar ainda mais a robustez da detecção da bola; - identificar automaticamente eventos (passes, finalizações e gols); - otimização do tempo da análise.

5. Implementação. `conversor-video/docs/seminario-iii/`: `seminario-iii-pitchview.pptx`, `roteiro-apresentacao.md`, `preview-slides.png`, `assets/` e `seminario-iii-conteudo-texto.pptx`.

6. Fluxo. Capturas reais (`localhost:3000`) ↓ geração do `.pptx` ↓ validação com `python-pptx` ↓ commit.

7. Problemas encontrados. O usuário estava pelo celular e precisou de instruções para baixar o arquivo pelo GitHub (link *raw*) ou pelos artefatos do agente (turno 15).

8. Por que essa solução. O texto final foi o definido pelo usuário. O deck visual ficou como alternativa.

9. Impacto. Nenhum no código. Os próximos passos do slide orientaram a fase seguinte (eventos).

10. Resultado. Deck de 5 slides com os próximos passos atualizados.

3.17 Estudo do repositório Simo-03 e proposta da camada de eventos (turnos 16 e 17, sem código)

1. Objetivo. Antes de implementar posse e passes, decidir se valia reaproveitar a lógica do repositório público `Simo-03/football-player-detection`, adaptá-la ou criar uma lógica própria.

2. Como funcionava antes. O pipeline tinha visão (bola, jogadores, tracking, times, homografia), mas nenhum evento de jogo. No resultado, `features` e `machine_learning` estavam como `pending`.

3. Qual era o problema. Como transformar sinais de visão em eventos confiáveis (posse, passe, troca de posse) sem criar falsos eventos por erro do tracker.

4. Solução adotada (análise).

Turno 16, o que o Simo-03 faz. - Arquitetura: YOLO11s + BoT-SORT + bola fatiada → times por K-Means em LAB → homografia RANSAC → posse por proximidade → passe/turnover por transição de possuidor → painel. - Posse em **pixels**: - jogador mais próximo da bola, a até 100 px; - confirma com 5 quadros, perde com 5, troca com 8; - a posse por time é a soma de quadros. - Passe: possuidor mudou, mesmo time, distância entre 3 m e 60 m. Turnover: times diferentes. - O README deles relata 7 passes e 2 turnovers em um clipe de 30 s. - Não há chute, gol, desarme nem bola dividida.

Conclusão do turno 16. Era viável adaptar sem mudar a arquitetura. Valia reaproveitar **só o contrato** de estados (histerese confirm/lose/switch, passe = transferência no mesmo time, turnover = transferência entre times, posse de time como soma de quadros). Visão, tracking, times e homografia deles não deveriam ser reaproveitados.

Turno 17, a proposta técnica própria ("BallEventLayer"). Uma camada analítica depois da homografia, sem mexer em RF-DETR, ByteTrack, TeamClassifier ou homografia: - **`BallState` por quadro:** `observed`, `remembered`, `predicted` ou `lost`. Só bola observada abre ou troca posse. Memória e previsão podem **manter** a posse, mas não abri-la. - **Possuidor:** só `Player`/`Goalkeeper` com `track_id ≥ 0`, medido pelos pés em **centímetros de campo**, com pixels só como fallback. Raio de controle sugerido de 1,8 a 2,5 m e de disputa de 2,5 a 4 m. - **Estados:** `NONE`, `PENDING(player)`, `CONTROLLED(player)` e `LOOSE`. - **Passe:** A controlado → a bola sai (release) → voo coerente → B do mesmo time controlado, em uma janela realista. A bola deve se deslocar de fato. - **Turnover:** a mesma máquina, com time diferente. Interceptação, desarme e recuperação de bola solta ficariam como especializações futuras. - **8 regras anti-falso-passe:** 1. fusão de identidade espacial (um track que some e renasce perto não gera passe); 2. o passe exige deslocamento da bola; 3. posse confirmada nos dois lados; 4. time estável; 5. homografia inválida reduz a confiança e não inventa evento; 6. bola só memorizada não fecha evento; 7. troca em 1 ou 2 quadros com distância grande é suspeita; 8. `track_id = -1` é ignorado.

5. Implementação. Nenhuma. Foram turnos de análise.

6. Fluxo proposto. detecção → ByteTrack → times → homografia → BallEventLayer (PossessionEngine → TransferEngine) → JSON de eventos.

7. Problemas encontrados. Na comparação apareceram as fragilidades do Simo-03 para o nosso caso: - posse em pixels varia com zoom; - "sumiu a bola → perdeu a posse"; - "mudou o ID → nasceu um evento"; - filtro rígido de 3 a 60 m como única validação.

8. Por que essa decisão. O nosso pipeline já tinha sinais mais ricos que o deles: coordenadas de campo, memória e previsão da bola, cache de times e histórico de movimento. Copiar a lógica deles jogaria fora esses sinais.

9. Impacto. Definiu a arquitetura das Seções 3.18 a 3.20. Parte das regras virou código mais tarde (bola observada como única fonte de abertura de posse, posse em campo, trajetória como evidência). A fusão espacial de IDs ficou pendente (Seção 5).

10. Resultado. Uma decisão de arquitetura: lógica própria, com o Simo-03 só como esqueleto de estados.

3.18 Painel "Análise da Partida" e contrato `events` — `3674ed2` (19/09/2026)

1. Objetivo. Preparar o frontend para posse e passes antes de existir a lógica. A seção "Análise da Partida" substituiria os dados técnicos durante a análise e mostraria: - posse atual; - passes do Time Azul e do Time Vermelho; - trocas de posse; - bola detectada ou não.

O contrato futuro era `events: { current_possession_team, blue_passes, red_passes, turnovers, ball_detected }`.

2. Como funcionava antes. O modal mostrava um rótulo técnico, jogadores detectados, frames processados, o diagrama YOLO/Tracking/ML e mensagens como "YOLO processou N frames".

3. Qual era o problema. Essas informações eram pouco úteis para o usuário final.

4. Solução adotada. Trocar a área técnica pelo painel da partida, com valores de placeholder (`null`/`0`/`false`, ou "Aguardando análise"). O backend passou a normalizar e entregar `events` sem calcular nada. O status visível ficou só "Analisando a partida..." ou "Análise concluída.".

5. Implementação. - `public/app.html`, `public/css/style.css`. - `public/js/analysisViewer.js` (`possessionLabel`, `getEvents`, `renderMatchEvents`). - `server/services/analysis/matchEvents.js` (novo: `normalizePossessionTeam`, `normalizeMatchEvents`, `createPlaceholderMatchEvents`, `toUserFacingAnalysisMessage`). - `AnalysisService.js`, `AnalysisProvider.js`, `MockAnalysisProvider.js`.

6. Fluxo. Worker emite progresso (ainda sem `events`) ↓ `normalizeMatchEvents` devolve o placeholder ↓ `getEvents` monta o objeto da tela ↓ `renderMatchEvents` escreve "Aguardando análise", 0, 0, 0 e "Detectada"/"Não detectada".

7. Problemas encontrados. Em `getEvents`, os passes e as trocas ficaram **fixos em 0** (linhas 80 a 82). Foi uma escolha correta para um placeholder, mas depois virou o motivo de passes "sumirem" do painel (Seções 3.31 e 3.33).

8. Por que essa solução. O usuário pediu explicitamente para separar interface e lógica, com um commit único.

9. Impacto. Só frontend e comunicação backend → frontend.

10. Resultado. O painel ficou pronto para receber posse, passes e trocas.

3.19 PossessionEngine: máquina de estados de posse em coordenadas de campo — `da50b21` (20/09/2026); merge `69e1d19`

1. Objetivo. Implementar **só** a posse de bola: robusta, em estados, usando o que já existia (bola, Ball Memory, predição, ByteTrack, TeamClassifier, homografia, `field_x/field_y`). Expor `events: { current_possessor_track_id, current_possession_team }` e mostrar a posse atual em tempo real.

2. Como funcionava antes. Não havia posse. O painel mostrava "Aguardando análise".

3. Qual era o problema. "Jogador mais próximo neste quadro" oscila entre jogadores próximos, perde a posse quando a bola some por poucos quadros e depende do zoom quando medido em pixels.

4. Solução adotada. - **Estado da bola** (`build_ball_state`): - `observed` quando há detecção real; - com a trilha interna viva (`internal_alive`) e sem detecção: `predicted` (se houver previsão em campo ou pixel) ou `remembered` (última bola); - trilha morta: `None`. - **Candidatos:** jogadores e goleiros com `track_id ≥ 0`, pelos pés. Distâncias em campo (cm) têm prioridade sobre pixels (`_rank_players` ordena primeiro por espaço e depois por distância). - **Raios** (`possession.py`):

	Campo	Pixel
Controle	250 cm (`CONTROL_RADIUS_CM`)	90 px
Retenção	400 cm (`HOLD_RADIUS_CM`)	140 px
Margem de disputa	70 cm (`CONTEST_MARGIN_CM`)	25 px

- **Histerese:**

Constante	Valor
`POSSESSION_CONFIRM_FRAMES`	5
`POSSESSION_SWITCH_FRAMES`	8
`POSSESSION_LOSE_FRAMES`	8

- **Regras:**
- só bola `observed` (`allow_new`) pode abrir ou trocar a posse;
- o dono atual se mantém enquanto estiver no raio de retenção, mesmo com bola lembrada ou prevista;
- se outro jogador estiver quase à mesma distância (dentro da margem de disputa), a posse não troca;
- a troca exige que o novo candidato lidere, dentro do raio de controle, por 8 quadros;
- a posse cai depois de 8 quadros sem evidência.
- **Time:** o `team_id` do possuidor vem do TeamClassifier por track (`_remember_teams`, `_refresh_possessor_team`). Na época era 0 → "blue" e 1 → "red".

5. Implementação. - `server/services/analysis/yolo/possession.py` (novo): - classe `PossessionEngine` com `update`, `_rank_players`, `_advance_confirm`, `_advance_switch`, `_lose_if_unsupported`, `_set_possessor`, `_refresh_possessor_team`, `_finish_update`, `snapshot` e `metrics` (`frames_with_possessor`, `confirmed_gains`, `losses`, `switches`); - `PossessionSnapshot`, `build_possession_players`, `build_ball_state`. - `run_yolo_analysis.py`: chamada logo depois da homografia. `events` é emitido no progresso, e a posse alterada força um novo `emit`. - `YoloAnalysisProvider.js`, `matchEvents.js`, `AnalysisService.js`, `AnalysisProvider.js`, `MockAnalysisProvider.js`, `analysisViewer.js`. - Teste `scripts/test-possession-engine.py`.

6. Fluxo. Quadro com jogadores (`track_id`, `team_id`, `field_x/field_y`) ↓ `build_ball_state` ↓ bola ausente ou perdida? `_lose_if_unsupported` (conta até 8) ↓ ordena os candidatos por distância ↓ **há dono:** está no raio de retenção? mantém (se não houver desafiante claro) ou avança a troca (8 quadros); fora do raio e com outro jogador no controle e bola observada? avança a troca; senão, conta a falta de evidência ↓ **não há dono:** bola observada e alguém no raio de controle? avança a confirmação (5 quadros) ↓ `_finish_update` soma `frames_with_possessor` ↓ `snapshot` ↓ emite `events`.

7. Problemas encontrados. - O clipe de teste é curto e a bola fica perto do mesmo jogador, então o teste só verificava estabilidade. - No merge com `main` (turno 20, `69e1d19`), dois arquivos tiveram conflito, todos simples: - `requirements.txt`: ficaram `rfdetr` e `huggingface_hub`; - `run_yolo_analysis.py`: 33 trechos; a branch era um superconjunto da memória/predição/ROI do PR #10.

8. Por que essa solução. É a proposta do turno 17. As distâncias em campo não dependem da câmera. A histerese evita oscilação, e a bola lembrada sustenta a posse sem inventar novos donos.

9. Impacto. Criou o snapshot de posse, que é a entrada do PassDetector, da posse por time e da reatribuição. Detecção, tracking e times não foram alterados.

10. Resultado. No clipe de 150 frames: - posse confirmada no quadro 7 para o track 10 (na época rotulado "red"); - **144 de 150** quadros com o mesmo possuidor; - 1 confirmação, 0 perdas, 0 trocas por oscilação.

3.20 PassDetector v1: confirmação de passe sobre snapshots de posse — `c7c8c6f` (PR #13, mesmo conteúdo de `c7d644a`) (21/09/2026)

1. Objetivo. Detectar passes em cima da posse, com filtros contra falsos positivos: - receptor com posse por pelo menos 5 quadros; - a bola de fato saiu do raio de retenção do passador; - intervalo entre perder e ganhar a posse de no máximo cerca de 1 s; - distância entre 2 m e 60 m; - direção da bola compatível; - sem duplicatas.

Estatísticas novas no perfil: `total_passes`, `discarded_short_passes`, `discarded_long_passes`, `discarded_unstable_possession`, `discarded_timeout` e `discarded_direction_check`.

2. Como funcionava antes. O arquivo `pass_detector.py` aparece pela primeira vez neste commit. Antes não havia contagem de passes.

3. Qual era o problema. Uma troca de possuidor do mesmo time pode ser passe, oscilação do tracker ou bola dividida.

4. Solução adotada. Um passe só entra quando **todas** as condições valem: - receptor B segura a posse por 5 quadros; - a bola saiu do raio de retenção do doador A; - intervalo de perder/ganhar ≤ 1 s; - distância entre 2 m e 60 m; - trajetória recente da bola apontando para o receptor; - o par de `track_id` não repete o evento anterior.

Também foi criado `raw_candidate_passes`, a contagem ingênua antes dos filtros.

5. Implementação. - `pass_detector.py` (novo, classe `PassDetector`). - `run_yolo_analysis.py`: chamada por quadro e métricas no perfil (`pass_detection`). - `scripts/test-pass-detector.py`: posse instável, raio, timeout, curto, longo, direção oposta e duplicata.

6. Fluxo. Snapshot de posse ↓ o possuidor mudou e o time é o mesmo? ↓ candidato bruto ↓ B confirmou 5 quadros extras? ↓ a bola saiu do raio de A? ↓ intervalo ≤ 1 s? ↓ 2 m ≤ d ≤ 60 m? ↓ a trajetória aponta para B? ↓ não é duplicata? ↓ passe.

7. Problemas encontrados. O clipe de 150 frames não tem troca de possuidor, então não foi possível medir falsos descartados no vídeo real. Os filtros foram validados só por testes sintéticos.

8. Por que essa solução. Era o refinamento mínimo sobre a posse, sem mexer nos outros módulos.

9. Impacto. Só o detector e o perfil. O painel continuava mostrando 0 passes (Seção 3.18).

10. Resultado. - Passes antes e depois: **0 → 0**. - Posse 144/150 no track 10. - Custo da etapa: **1,7 ms no total** (0,011 ms por quadro). - Tempo de parede: 179,4 s contra 190,1 s na corrida anterior (ruído entre execuções).

Limitação descoberta depois (Seção 3.21): os filtros extras (5 quadros + raio + 1 s + trajetória) cortariam toques rápidos. Isso levou à reescrita por placar (Seção 3.29).

3.21 Diagnóstico técnico geral (turno 22, sem código)

1. Objetivo. Antes de corrigir, entender os problemas vistos na última análise: - bola visível, inclusive laranja, não detectada; - posse para o time errado; - passes claros perdidos; - bandeirinha como jogador e árbitro ↔ jogador trocados; - trocas de classe e IDs instáveis; - lentidão.

2. Como funcionava antes. O pipeline estava como nas Seções 3.10 a 3.20.

3. Qual era o problema. Muitos sintomas e nenhuma ordem de correção definida.

4. Solução adotada (diagnóstico por problema).

#	Problema	Causa provável	Confiança	Módulo	Correção recomendada	Prioridade
1	Bola não detectada	O backend efetivo era `yolo`. Predição e busca híbrida só ligam com `rfdetr`. O modelo de bola é fraco para bola laranja ou pequena. O limiar longe é 0,35. A validação física rejeita saltos > 300 px.	Alta (backend); média (bola laranja)	`run_yolo_analysis.py` + `football-ball-detection.pt`	Backend melhor por padrão, limiar menor perto da trilha, memória que não esconda bola nova	1
2	Posse no time errado	Não era a máquina de posse. O TeamClassifier sem `random_state` permutava os clusters entre execuções (track 10 saiu "red" numa corrida e "blue" em outra). O refresh de 30 quadros podia inverter o time no meio do vídeo.	Alta	`roboflow_team.py` + `_refresh_possessor_team`	Travar o time por track e ancorar o significado do cluster	2
3	Passes perdidos	Consequência da bola e da posse. No clipe, `raw_candidate_passes=0`. Os filtros extras do detector seriam secundários.	Alta (upstream); média (filtros)	Bola + posse; depois `pass_detector.py`	Não afrouxar filtros antes de corrigir bola e time	4
4	Bandeirinha / árbitro como jogador	O detector errou a classe (0 árbitros e 0 goleiros em 150 quadros). O filtro espacial era código morto (`accepted_player_detections = list(player_detections)`).	Alta / média-alta	Detector de pessoas + filtro desligado	Rejeitar objetos sem forma de pessoa; travar a classe	3
5	Classe e ID instáveis	ByteTrack associa caixas, não classe. A classe era votada em 9 quadros. IDs novos depois de oclusão.	Alta (mecanismo)	ByteTrack + `stabilize_player_referee_class`	Congelar a classe por track; reassociação espacial	5
6	Lentidão	Três inferências completas por quadro em CPU: jogadores 26,1%, campo 26,3%, bola 25,3%; TeamClassifier fit 7,9% + predict 5,6%. 179 s para 6,1 s de vídeo (~0,84 FPS). Posse e passes são irrelevantes no tempo.	Alta	Inferências	GPU; pitch a cada N quadros; `imgsz` menor só na bola	3 (para conseguir iterar)

Ordem recomendada: bola → TeamClassifier → classe Player/Referee/bandeira → passes → tracking → desempenho.

5. Implementação. Nenhuma.

6. Fluxo do diagnóstico. Leitura do perfil e do resultado do clipe ↓ comparação entre execuções ↓ leitura do código de cada módulo ↓ hipótese, evidência e confiança ↓ prioridade.

7. Problemas encontrados. A falha da bola laranja não aparecia no clipe de 150 frames (150/150 detecções). O diagnóstico separou o que foi medido do que era causa estrutural para vídeos mais longos.

8. Por que essa ordem. Os passes dependem da bola e da posse. O tracking depende da classe. Afrouxar o detector de passes antes disso mediria ruído.

9. Impacto. Orientou as Seções 3.22 a 3.28.

10. Resultado. Um mapa de causas com prioridades. O desempenho (pitch a cada N quadros, GPU) ficou pendente.

3.22 TeamClassifier estável: âncora de cor da camisa + lock por track — diagnóstico (turno 23) e `1fb0cef` (22/09/2026; originalmente `945ba0d`)

1. Objetivo. Resolver dois problemas: 1. o mesmo `track_id` mudava de time **durante** a análise; 2. o mesmo vídeo gerava "Azul" numa execução e "Vermelho" em outra.

A restrição era não alterar RF-DETR, ByteTrack, BallTracker, predição, homografia, detector de passes nem frontend.

2. Como funcionava antes. - Fit único antes da análise: a cada 10 quadros, 60 recortes `Player` (no clipe, 50 quadros e 60 recortes). - SigLIP → UMAP 3D → KMeans k = 2, **sem** `random_state`. - Cache por track que **repredizia e sobrescrevia** o time a cada 30 quadros. - A posse republicava o time a cada quadro. - Havia fallback `team_id = 0` quando o vetor de predição vinha curto.

3. Qual era o problema (diagnóstico do turno 23). - **Entre execuções:** KMeans é invariante a permutação. Sem semente, os rótulos 0/1 trocam de lugar. UMAP também é estocástico. Evidência: o track 10 saiu "red" e depois "blue" no mesmo vídeo, com 1 ganho e 0 trocas na posse. - **Durante a partida:** no quadro 31, 61, 91…, o recorte atual era reclassificado e **substituía** o time: - `UMAP.transform` é aproximado e coloca pontos perto da fronteira; - o recorte muda com pose, iluminação e caixa; - árbitro e bandeira entravam no fit (0 árbitros detectados), sujando os dois clusters (predição de 10 contra 4 no clipe); - havia o fallback para 0; - a posse publicava o novo time imediatamente. - **Fixar `random_state` não bastava:** isso reproduz o mesmo fit com os mesmos recortes, mas não dá significado a 0/1, não impede a inversão no meio do jogo e não limpa os recortes.

4. Solução adotada. 1. **Âncora de cor (`anchor_cluster_labels`):** - depois do fit, calcular o perfil de cor do **torso** de cada cluster (`torso_region`: 22–62% da altura e 22–78% da largura); - `jersey_color_profile` / `jersey_blueness` = azul − vermelho + 0,002 × croma; - o cluster mais azul vira o time 0 (`TEAM_BLUE_ID`) e o outro, o time 1, qualquer que seja o número sorteado pelo KMeans. 2. **Lock por track (`TeamTrackLock`):** - o time trava depois de **3 classificações iguais** (`TEAM_LOCK_CONFIRM_STREAK`); - o refresh de 30 quadros passa a ser só **voto de verificação**; - só **3 refreshes consecutivos discordantes** (`TEAM_LOCK_UNLOCK_STREAK`) destravam. 3. **Fit limpo (`reject_training_crop`):** antes do SigLIP, do UMAP e do KMeans, descarta recortes: - pequenos (< 40 × 18); - finos sem forma de jogador (`thin_non_player`, aspecto < 0,18); - largos (`wide_non_player`, > 1,05); - com predominância de grama (`mostly_grass`, > 72%); - bandeirinha (`corner_flag`, aspecto < 0,32 com > 22% de matiz de bandeira no terço superior); - oficiais escuros de baixa saturação (`low_chroma_official`, > 62% de torso neutro escuro). 4. Semente fixa (`random_state=0`) no UMAP e no KMeans (`n_init=10`) como complemento.

5. Implementação. - `roboflow_team.py`: `torso_region`, `jersey_color_profile`, `jersey_blueness`, `anchor_cluster_labels`, `reject_training_crop`, classe `TeamTrackLock`, e `TeamClassifier` com a âncora. - `run_yolo_analysis.py`: usa o lock em vez de sobrescrever o cache; novas métricas no resultado. - Teste `scripts/test-team-classifier-stability.py`.

6. Fluxo. Pré-passagem: recortes de jogador ↓ `reject_training_crop` ↓ SigLIP ↓ UMAP (seed 0) ↓ KMeans (seed 0) ↓ perfil de cor do torso por cluster ↓ mais azul = time 0 ↓ análise: track novo?

classifica e acumula a sequência; 3 iguais? trava ↓ track travado: a cada 30 quadros, verifica; concorda? zera as contradições; discorda? soma; 3 seguidas? destrava e reclassifica ↓ a posse lê o time travado.

7. Problemas encontrados. - O turno 23 descartou "congelar o primeiro predict sem âncora", porque o time ficaria estável, mas invertido, a partida inteira. - Também descartou "refitar o KMeans no meio do vídeo", porque reabre a permutação. - Remover o UMAP foi considerado opcional e não foi feito.

8. Por que essa solução. A âncora dá **significado** ao rótulo (a camisa), o que a semente sozinha não dá. O lock elimina a inversão no meio da partida. Limpar o fit corrige a fronteira.

9. Impacto. A posse e os passes passaram a herdar um time estável. O detector de passes não foi alterado.

10. Resultado. No clipe de 150 frames, com duas execuções novas: - o mesmo mapa de times nas duas, com o track 10 "Azul" nas duas; - a âncora remapeou o cluster cru `{0 → vermelho, 1 → azul}` nas duas execuções; - 14 tracks travados; 2 trocas antes do lock; **1 troca depois do lock** (track 7, de Vermelho para Azul no quadro 101, depois de 3 refreshes discordantes); - 56 verificações, com 53 concordâncias e 3 contradições; - `discarded_non_player.mostly_grass = 10`; - tempo de 169 s e 168 s (fit ~11,5 s, predict ~12,4 s) contra 179 s antes.

Métricas novas: `color_anchor.cluster_to_team`, `labels_were_remapped`, `stability.*` (`tracks_seen`, `tracks_locked`, `prelock_team_switches`, `team_changes_after_lock`, `refresh_checks`, `refresh_agreements`, `refresh_contradictions`, `unlocks`), `final_team_by_track` e `discarded_non_player`.

3.23 Nomenclatura Team 0 / Team 1 — `effbbde` (22/09/2026; originalmente `9a29e53`)

1. Objetivo. Remover "Time Azul" e "Time Vermelho" de todo o pipeline (posse, eventos, métricas, logs, JSON e frontend) e usar só "Team 0" e "Team 1", alinhado aos IDs do classificador e do tracking. Só nomenclatura, sem mudar lógica.

2. Como funcionava antes. - A posse publicava `blue`/`red`. - `matchEvents.js` normalizava para `blue`/`red`. - O frontend mostrava "Time Azul"/"Time Vermelho".

3. Qual era o problema. Com a âncora de cor, o time 0 é o "mais azul". Ainda assim, chamar os times de Azul e Vermelho assume as cores dos uniformes, o que não vale para qualquer jogo.

4. Solução adotada. - `TEAM_NAME_BY_ID = {0: "Team 0", 1: "Team 1"}`. - Eventos e JSON com `"Team 0"` e `"Team 1"`. - `color_anchor.semantic` com a mesma convenção. - Painel com "Team 0" e "Team 1".

5. Implementação. `possession.py`, `pass_detector.py`, `roboflow_team.py`, `matchEvents.js`, `app.html`, `analysisViewer.js`, `test-pass-detector.py`, `test-possession-engine.py`.

6. Fluxo. `team_id` 0/1 ↓ `TEAM_NAME_BY_ID` ↓ `"Team 0"`/`"Team 1"` no snapshot ↓ `normalizePossessionTeam` ↓ `possessionLabel` no painel.

7. Problemas encontrados. No turno 37 o usuário viu "Time Azul" na tela, mesmo estando no commit mais recente (`c7c8c6f`). A investigação mostrou que a renomeação existia só no commit `9a29e53`, na branch `cursor/team-nomenclature-5e9e`, que não tinha sido incorporada. Isso motivou a auditoria (Seção 3.30).

8. Por que essa solução. Consistência entre classificador, tracking, eventos e interface.

9. Impacto. Só textos e contratos de nome. Mais tarde virou um conflito de intenção com `main`, que usa `blue`/`red` (Seção 3.34).

10. Resultado. Posse, passes, eventos e painel usam Team 0 e Team 1.

3.24 PersonClassLock: classe Player / Referee / Goalkeeper travada por track — discussão (turno 26) e `e88f3ec` (22/09/2026; originalmente `646c698`)

1. Objetivo. - Impedir que um jogador vire árbitro, que um árbitro vire jogador e que uma bandeirinha, poste ou objeto fino seja tratado como jogador. - O árbitro travado não pode participar do TeamClassifier, da posse nem dos passes.

2. Como funcionava antes. - A classe vinha do detector, com voto de maioria em 9 quadros (`stabilize_player_referee_class`). - Não havia filtro de não-pessoa, porque o filtro espacial tinha sido removido (Seção 3.13).

3. Qual era o problema. - Objetos finos entravam como jogador, ganhavam `track_id` e podiam "pegar" a bola na posse. - O voto de 9 quadros só atrasava a troca de classe.

No turno 26 o usuário propôs uma ordem de prioridades: classes → bola → posse → passes → estatísticas. A análise concordou e acrescentou: "Para o lock funcionar, a classe precisa nascer certa": rejeitar objetos finos **antes** de criar o track. Senão, o lock congela o erro.

4. Solução adotada. - **Antes do ByteTrack**, `reject_non_person_detections` descarta caixas por `non_person_rejection_reason`: - aspecto < 0,16: `thin_non_person`; - aspecto > 1,8: `wide_non_person`; - aspecto < 0,24 com mais de 45% de matiz de bandeira (vermelho, laranja ou amarelo saturado) no terço superior: `corner_flag`. - **`PersonClassLock`:** - trava depois de 3 quadros iguais (`PERSON_CLASS_CONFIRM_STREAK`); - depois do lock, um quadro discordante não muda nada; - 3 discordâncias consecutivas (`PERSON_CLASS_UNLOCK_STREAK`) destravam. - O árbitro travado fica fora do fit e da predição de time, da posse e dos passes.

5. Implementação. - `person_class.py` (novo): `non_person_rejection_reason`, `_top_is_corner_flag`, `reject_non_person_detections`, `PersonClassLock` (`assign`, `metrics`). - `run_yolo_analysis.py`: rejeição antes do ByteTrack, depois da separação da bola do modelo de jogadores; `assign` por track; roteamento por classe. - Teste `scripts/test-person-class-lock.py`.

6. Fluxo. Detector de jogadores ↓ separa a classe bola ↓ `reject_non_person_detections` ↓ ByteTrack ↓ para cada track, `PersonClassLock.assign(track, classe_crua)`: sem lock? conta a sequência e trava em 3; com lock? verifica; discordância 1 ou 2: mantém; 3: destrava ↓ Player/Goalkeeper seguem para times e posse; Referee fica de fora.

7. Problemas encontrados. O clipe de 150 frames não tem bandeirinha nem oscilação de classe. Esses casos só puderam ser validados nos testes sintéticos.

8. Por que essa solução. É a mesma lógica de lock com destravamento por evidência que funcionou no TeamClassifier. Rejeitar antes do tracking evita gastar IDs com objetos que não são pessoas.

9. Impacto. Afeta o conjunto de pessoas que alimenta times, posse e passes. RF-DETR, BallTracker, predição, homografia, TeamClassifier, posse, passes e frontend não foram alterados.

10. Resultado. No clipe:

	Antes	Depois
Trocas cruas Player/Referee	0	0
Trocas depois do lock	0	0

	Antes	Depois
Falsos jogadores rejeitados	—	0
Tempo	169,3 s	169,9 s

14 tracks travados como Player, 2030 verificações e 0 contradições. A posse continuou no track 10, Team 0, em 144/150 quadros.

3.25 Diagnóstico da perda da bola (turno 28) e "passe por buraco de posse" (PR #17, `2732558` / `e161b65`, não integrado)

1. Objetivo. - (a) Descobrir em qual etapa uma bola claramente visível desaparece. - (b) Registrar um passe quando a bola some por até 1 s entre a posse de A e a posse confirmada de B, do mesmo time.

2. Como funcionava antes. O detector v1 exigia trajetória e filtros rígidos.

3. Qual era o problema. Passes reais se perdiam porque a bola não era vista durante o trajeto.

4. Solução adotada.

(a) Diagnóstico do turno 28. O caminho que realmente roda com o backend `yolo`: 1. YOLO de bola no quadro inteiro, `imgsz=640`, confiança mínima 0,25; 2. NMS com IoU 0,1; 3. seleção pela previsão: perto (≤ 80 px) exige ≥ 0,12; longe, ≥ 0,35; 4. validação física: até 120 px aceita; de 120 a 300 px exige 2 quadros; acima de 300 px descarta; 5. `BallTracker`; 6. memória de 5 quadros, que não desenha e limpa o histórico no 6º quadro.

Ranking dos gargalos: 1. **Validação física**, se o perfil mostrar rejeições nos quadros com bola nítida. 2. **YOLO a 640 com corte de 0,25**, se nem a 0,05 a bola aparece. 3. Limiar de 0,35 longe da previsão. 4. Memória que não desenha e zera o histórico. 5. NMS a 0,1. 6. BallTracker. 7. Smart Crop, busca híbrida e ROI preditiva, que não rodam com `yolo`.

(b) Turno 29. Um passe do mesmo time é registrado pela máquina de posse quando a bola some por até 1 s e reaparece com B confirmado. Posições previstas ou lembradas não entram na trajetória.

5. Implementação. (b) foi feito na branch `cursor/possession-gap-passes-5e9e` (PR #17), em `pass_detector.py` e no teste. **Não foi integrado** à branch oficial: no turno 40 o usuário decidiu descartar `2732558` e `e161b65` por serem uma versão intermediária, substituída pelo placar de posse (Seção 3.29).

6. Fluxo (b). A tem posse ↓ posse vazia (≤ 1 s) ↓ B do mesmo time confirma ↓ filtros (2–60 m, 5 quadros, raio, duplicata) ↓ passe.

7. Problemas encontrados. - No clipe de 150 frames a regra não mudou nada: a posse fica vazia só nos 6 primeiros quadros e depois fica com um único jogador (track 2, time azul nessa execução). - Passes e turnovers ficaram 0 → 0, com tempo de 420,05 s e etapa de passes de 2,55 ms. - Nos testes, o caso "bola reaparece com B e a posição prevista deriva para longe" passou de 0 para 1 passe.

8. Por que foi substituída. A filosofia evoluiu para "posse confirmada como evidência principal e trajetória só como ajuste de confiança" (Seção 3.29).

9. Impacto. Nenhum na branch oficial.

10. Resultado. O diagnóstico (a) orientou as Seções 3.26 a 3.28.

3.26 Recall da bola fraca que continua a trilha (duas versões) — turno 30 (v1, limiar 0,12), `78a24f2` e `a503632` (v2, limiar 0,02) (27/09/2026; originalmente `0a21239`, `4f81770`)

1. Objetivo. Corrigir **só a maior causa** de falso negativo da bola, sem aumentar muito os falsos positivos e sem mexer em ByteTrack, TeamClassifier, posse, passes e frontend.

2. Como funcionava antes. O YOLO de bola só devolvia caixas com confiança ≥ 0,25.

3. Qual era o problema. - **Versão 1 (turno 30, clipe de 150 frames):** medindo cada etapa, a bola estava em 150/150 quadros com confiança média de 0,71. Havia uma caixa de cerca de 0,17 parada a ~260 px, que não era a bola. Validação física: 150 aceites e 0 rejeições. Memória: 0 usos. Conclusão: o corte de 0,25 do próprio YOLO era o ponto onde uma bola fraca sumiria. Nesse clipe isso não acontecia. - **Versão 2 (turno 31, gravação de tela com bola laranja, 401 quadros):** - com limiar 0,01 para listar tudo, a trilha da bola de 165 a 276 tem **112 quadros**; - **48** com caixa ≥ 0,25; - **41** com caixa sobre a bola **abaixo de 0,25** (valores de 0,010 a 0,223); - **23** sem nenhuma caixa até 45 px; - **3** (254, 255, 257) com caixa forte (0,281, 0,281, 0,430) segurada pela validação física; - no caminho de produção, a bola aparecia em só **45** dos 112 quadros; - em 5 quadros (184, 196, 197, 227, 252) havia uma caixa forte **longe** da bola (178 a 320 px). Ficar só com a caixa forte levaria a trilha para o falso positivo.

4. Solução adotada. - **v1:** aceitar caixas de 0,12 a 0,25 só quando continuam a bola seguida (≤ 80 px da última posição). Uma caixa ≥ 0,25 tem prioridade. Caixa fraca não abre trilha. - **v2 (`78a24f2`):** - o YOLO de bola passa a devolver caixas a partir de **0,02** (`BALL_YOLO_RECALL_CONFIDENCE`); - `keep_recoverable_ball_candidates` monta a lista de candidatas: 1. se houver caixa **forte** (≥ 0,25, `BALL_STRONG_CONFIDENCE`) a até **120 px** da previsão (`BALL_NEARBY_STRONG_DISTANCE_PX`), só elas; 2. senão, as caixas **fracas** (≥ 0,02) a até **80 px** (continuação); 3. senão, as fortes distantes; 4. sem previsão, só as fortes (caixa fraca não abre trilha); - uma sequência só de caixas fracas (`weak_yolo_continuation`) não prolonga a memória além dos 5 quadros. Depois disso, histórico e movimento são limpos, para que um falso positivo na borda não esconda a bola forte que aparece em seguida. - **`a503632`:** a caixa fraca na trilha é **preferida** a uma caixa forte distante (o caso dos quadros 184, 196, 197, 227 e 252).

5. Implementação. - `run_yolo_analysis.py`: constantes `BALL_YOLO_RECALL_CONFIDENCE = 0.02`, `BALL_STRONG_CONFIDENCE = 0.25` e `BALL_NEARBY_STRONG_DISTANCE_PX = 120`; função `keep_recoverable_ball_candidates`, aplicada depois do NMS e também nas listas semi-global e global; seleção com limiares 0,02 perto e 0,25 longe para o YOLO; `weak_yolo_continuation`. - Teste `scripts/test-ball-recall-filter.py`.

6. Fluxo. YOLO (conf ≥ 0,02) ↓ NMS ↓ há previsão? não: só caixas ≥ 0,25; sim: forte ≤ 120 px? fica com ela; senão, fraca ≤ 80 px? continuação; senão, forte distante ↓ seleção ↓ validação física ↓ a caixa é fraca? conta como continuação fraca; mais de 5 seguidas? limpa o histórico.

7. Problemas encontrados. - A v1 foi validada num vídeo em que a bola já era detectada em 150/150 quadros. O usuário apontou isso (turno 31) e exigiu o vídeo problemático. Isso levou à medição quadro a quadro e ao limiar de 0,02, porque muitas caixas reais ficavam abaixo de 0,12. - Na v1 o tempo foi de 420,05 s para 408,02 s, só variação de execução.

8. Por que essa solução. Baixar o limiar **global** abriria falsos positivos em todo o quadro. Restringir a caixa fraca à vizinhança da trilha recupera a bola real sem abrir trilhas novas.

9. Impacto. Só a seleção da bola. Mais tarde virou um dos conflitos de intenção com `main`, que usa a lista crua e o limiar de 0,25 (Seção 3.34).

10. Resultado (v2, mesma trilha de 112 quadros): - bola em **71** quadros, antes 45; - continuam sem bola 41: 23 sem caixa, 11 com caixa < 0,02 e 7 perdidos pela memória ou pela validação física; - nenhum quadro da trilha trocou a bola por uma caixa distante; - no vídeo inteiro, as seleções passaram de **130 para 209**; - três quadros isolados (67, 379, 382) deixaram de ser selecionados; - custo do detector a 0,02: 125,2 s em 401 quadros (312 ms por quadro).

3.27 Investigação dos 23 quadros sem caixa e estratégias para "o detector enxergar" (turnos 32 e 33, sem código)

1. Objetivo. Entender os 23 quadros em que o YOLO não gera caixa alguma e descobrir se alguma técnica faz o detector realmente ver a bola, sem heurísticas como interpolação, ponte ou memória.

2. Como funcionava antes. Situação da Seção 3.26.

3. Qual era o problema. Em 23 quadros a bola estava visível e não havia nenhuma caixa a até 45 px, nem com limiar 0,01.

4. Solução adotada (investigação).

Turno 32, padrão comum. - São **buracos de 1 a 3 quadros** na mesma bola laranja, com o mesmo tamanho. - A bola mede 10 a 15 px (mediana 13 px). Depois do *letterbox* para 640, fica com cerca de **6 px**, menor que uma célula de *stride* 8. - Em **10** desses quadros a caixa existe, mas com confiança de 0,0011 a 0,0041. - **7** quadros são cópias pixel a pixel do anterior (192, 206, 218, 233, 271, 272, 274), por causa da gravação de tela. - Frame 245: a mancha laranja cai pela metade (bola sobre um jogador). - Frames 269 a 276: um intervalo longo depois da última detecção (268).

Hipóteses e evidências:

Hipótese	Conclusão
Tamanho no 640	Explica 15 quadros (confiança alta)
Caixa < 0,01	Explica 10 quadros, todos já entre os 15
Duplicatas	Não recuperam nada
`imgsz=1280`	Recupera só 232, 233 e 245, com cerca de 4× o tempo
Recorte de 192 px	0 quadros
Oclusão	1 quadro (245)
Blur, velocidade, cor, iluminação, distância, compressão	Não separam os 23 dos 89 detectados

Turno 33, medição das alternativas (recuperação = caixa a até 30 px da bola):

Estratégia	Caixa ≥ 0,25	≥ 0,05	Tempo
Modelo de bola, 640	0/23	0/23	254 ms/quadro
Modelo de bola, 960	4/23	4/23	~2,4×
Modelo de bola, 1280	0/23	1/23	~4×
Nitidez + 1280	4/23	5/23	~4×
Tiles de 640 (6 por quadro)	0/23	2/23	~10×
Tiles de 384 (15 por quadro)	1/23	3/23	~26×
Classe `ball` do detector de jogadores (1280)	**15/23**	**16/23**	**já roda**

- A classe `ball` do `football-player-detection.pt`, que o pipeline já executava em 1280 e **descartava**, acertou 15 quadros com confiança de 0,29 a 0,82 e caixa de 10 a 12 px sobre a bola.
- Falsos positivos dessa classe no vídeo inteiro: média de 1,19 caixas por quadro, contra 0,29 do modelo dedicado.

- Na trilha já aberta, ela coincidiu com o modelo dedicado em 28 de 30 quadros. As caixas extras ficavam a centenas de pixels (x ≈ 1040–1240), e um portão de distância as segura.
- RF-DETR não foi medido porque os pesos não estavam no ambiente.
- Ajuste fino do modelo e super-resolução foram considerados de dificuldade alta ou de ganho baixo.

5. Implementação. Nenhuma.

6. Fluxo da investigação. Lista dos 23 quadros ↓ reexecução a 0,001, a 1280 e em recorte ↓ medição de cor, brilho, passo e duplicatas ↓ ranking de hipóteses ↓ comparação de estratégias com tempo e falsos positivos.

7. Problemas encontrados. - O usuário pediu explicitamente **não** usar ponte, interpolação ou memória (turno 33). Por isso a "ponte de 3 quadros", primeira colocada no turno 32 com 15/23, foi deixada de lado. - Os quadros 270 a 276 não têm caixa perto da bola em nenhuma opção. O detector de jogadores marca uma bola falsa na borda direita (x ≈ 1000, 0,47–0,69).

8. Por que a recomendação. Recuperava o maior número de quadros (15/23) com custo praticamente zero, porque o *forward* em 1280 já acontecia. O risco de falso positivo era controlável com o portão de distância.

9. Impacto. Levou à Seção 3.28.

10. Resultado. Uma estratégia escolhida com números medidos. O quadro 245 só entraria com corte de 0,20, e os quadros 270 a 276 continuaram sem solução.

3.28 Fallback da bola pela classe `ball` do detector de jogadores — `d06e77a` (PR #19) (27/09/2026; originalmente `b3bd8e0`)

1. Objetivo. Usar a classe `ball` do detector de jogadores **só** quando o detector dedicado não devolve uma bola válida, e **só** para continuar uma trilha existente. Restrições: não alterar RF-DETR, ByteTrack, TeamClassifier, PassDetector, PossessionEngine, frontend nem a lógica do detector dedicado.

2. Como funcionava antes. A classe 0 do detector de jogadores era descartada. As pessoas seguiam para o ByteTrack.

3. Qual era o problema. 15 dos 23 quadros sem caixa poderiam ser recuperados com uma inferência que já existia (Seção 3.27).

4. Solução adotada. - A classe 0 do detector de jogadores vira `player_ball_candidates`. As caixas de pessoa enviadas ao ByteTrack continuam as mesmas. - **Depois da validação física e antes do `BallTracker`**, se o detector dedicado não produziu bola, `select_player_ball_fallback`: - exige uma trilha e uma previsão em pixels (sem elas, nunca abre trilha); - aceita só caixas a até **80 px** da previsão e com confiança ≥ **0,25**; - escolhe a mais próxima e, em empate, a de maior confiança; - marca a origem como `player-detector-ball`. - Quando o fallback continua a trilha, a candidata física distante ainda pendente é descartada.

5. Implementação. - `run_yolo_analysis.py`: `select_player_ball_fallback`, separação da classe 0, contadores de perfil `ball_source` (principal / fallback / nenhum) e `ball_fallback` (considerados, usados, rejeitados por distância ou confiança, confiança média, distância média). - Teste `scripts/test-player-ball-fallback.py`.

6. Fluxo. Detector de jogadores ↓ classe 0 vai para `player_ball_candidates` ↓ detector dedicado ↓ seleção ↓ validação física ↓ o detector dedicado deu bola? usa ↓ não: há trilha e previsão? ↓ candidata ≤ 80 px e ≥ 0,25? ↓ usa (origem fallback) ↓ BallTracker.

7. Problemas encontrados. - O contador de rejeição por confiança ficou em 0, porque o detector de jogadores já roda com `conf=0.25`. - **Turno 35:** o usuário pediu a precisão dos 89 quadros do fallback. Não foi possível calcular: - a gravação não estava mais no ambiente; - o perfil guardava só agregados, sem a lista de quadros; - o único vídeo presente era o clipe de Barcelona × Osasuna, que é outro jogo.

Registrou-se só o que tinha sido visto: os quadros 32, 241 e 269 estão corretos (bola laranja), e o 270 foi rejeitado (linha lateral, 0,69, a 552 px).

8. Por que essa solução. Recupera a bola real com custo zero e com o mesmo portão espacial que já protegia a trilha.

9. Impacto. - Etapa da bola. - A posse e os passes ganharam evidência: posse em 106 → 182 quadros e o primeiro passe real. - A validação física teve contadores alterados, mas a função não mudou.

10. Resultado (gravação de 401 quadros):

	Antes	Depois
Quadros com bola	202	302
Detector principal	202	213
Fallback	0	89
Nenhum dos dois	—	99
Quadros com posse	106	182
Passes	0	1
Validação física (aceites / rejeições)	202 / 26	213 / 28
Tempo total	1153 s	1047 s

- `ball_fallback`: 188 considerados, 89 usados, 73 rejeitados por distância, confiança média 0,62, distância média 10,8 px.
- Dos 23 quadros, 15 foram recuperados, cada um a no máximo 1,8 px da bola conhecida.
- O 245 (0,21) e os quadros 270 a 276 ficaram de fora.
- Nenhuma bola aceita caiu na lateral: nas 82 aceitações do replay, a distância máxima foi 45 px (quadro 32, que é a bola).
- No quadro 241, a bola de 0,65 foi usada e a de 0,75 na linha lateral, ignorada.
- A queda de tempo veio da carga da máquina, não do fallback.
- Posse: 7 ganhos e 7 perdas, contra 4 e 4. Passes: 5 candidatos brutos, 2 descartados por tempo, 1 por distância curta e 1 por posse instável.

3.29 PassDetector v2: placar orientado à posse — `b47da34` (PR #20) (04/10/2026; originalmente `ea7408a`)

1. Objetivo. Reescrever o detector com a filosofia de que a informação mais confiável é a **posse confirmada**: possuidor, time, `track_id` estável e instante. A trajetória da bola vira só evidência auxiliar. O detector deve detectar quem passou e quem recebeu mesmo sem ver a trajetória, e classificar o lance como **passe, turnover, disputa ou condução**. Nada fora do detector podia ser alterado.

2. Como funcionava antes (v1). - O passe só fechava depois de mais 5 quadros com B. - A bola precisava sair do raio de A. - Uma trajetória completa na direção errada eliminava o evento. - A mesma dupla A → B não podia contar de novo.

3. Qual era o problema. Passes rápidos, oclusões e bolas pequenas fazem a trajetória falhar. Na v1, a falta de bola matava o passe.

4. Solução adotada. - **Sequência:** A com posse confirmada ↓ posse vazia (opcional, até 1 s) ↓ B aparece no snapshot. O snapshot de B já é confirmação, porque a máquina de posse só publica depois dos quadros de confirmação dela. - **Classificação:** - **condução**, se B = A; - **disputa**, se os dois times ficaram a até **1,2 m** da bola observada antes da confirmação (`PASS_CONTEST_RADIUS_CM = 120`; 45 px em pixel); - **turnover**, se os times são diferentes e não houve disputa; - **passe**, se o time é o mesmo e o placar chega a **80**. - **Placar:**

Componente	Pontos
Mesmo time	+40
Mudança de possuidor	+30
Intervalo ≤ 1 s	+10
Distância plausível (1,5 m a 70 m)	+10
Trajetória observada compatível	+10
Trajetória observada incompatível	−20
Bola ausente	0

- Sem bola, o placar fica em 90 (distância plausível) ou 80 (distância não mensurável). Os dois passam.
- Trajetória contrária dá 70, que vira "candidato não confirmado" (`unconfirmed_candidates` / `unconfirmed_events`).
- **Cortes antes do placar:** < 1,5 m (`PASS_MIN_DISTANCE_CM = 150`), > 70 m (`PASS_MAX_DISTANCE_CM = 7000`) e intervalo > 1 s.
- Posição prevista ou lembrada **não entra** na trajetória.
- **Turnover** com confiança 70, +10 se a trajetória é compatível e −10 se é incompatível (`_turnover_confidence`).

5. Implementação. - `pass_detector.py` reescrito. - A interface `update(frame, fps, snapshot, players, ball)` e os contadores lidos pelo perfil foram preservados. - Novos: `turnovers`, `disputes`, `conductions`, `unconfirmed_candidates`, `pass_score_threshold`, `events`, `unconfirmed_events`, `turnover_events`, `dispute_events`. - `run_yolo_analysis.py`: 6 linhas para expor as métricas. - `test-pass-detector.py`: passe com bola sumida, trajetória oposta abaixo do limiar, timeout de 1 s, distância de 0,8 m e de 80 m, condução, disputa e turnover.

6. Fluxo. Possuidor A confirmado ↓ posse vazia: abre `pending` (`from_id`, `from_team`, `from_pos`, `from_frame`, `lost_frame`, `ball_missing_frames`, `contested`) ↓ a cada quadro: bola ausente? `ball_missing_frames++`; os dois times na bola? `contested = True` ↓ B confirma: B = A? condução; disputa? `dispute_events`; times diferentes? turnover; time desconhecido? `discarded_unstable_possession` ↓ mesmo time: cortes de tempo e distância ↓ placar ↓ ≥ 80? `events`; senão, `unconfirmed_events`.

7. Problemas encontrados (riscos registrados). - O detector **herda os erros da posse**: um snapshot errado gera um passe errado com confiança média. - Uma marcação a menos de 1,2 m no momento da soltura vira disputa, e o passe não é registrado. - Sem coordenadas de campo, a distância não elimina absurdos.

8. Por que essa solução. A posse confirmada é mais estável que a trajetória da bola. O placar troca decisões binárias por evidência graduada, e a ausência de bola deixa de matar o passe.

9. Impacto. Só `pass_detector.py`. O `PossessionEngine`, os detectores, o ByteTrack e o frontend não foram alterados.

10. Resultado. Detector por placar, com passe, turnover, disputa e condução. Na integração (Seção 3.30), o clipe de 150 frames continuou com 0 passes, porque não há troca de possuidor.

3.30 Auditoria e integração das branches paralelas na branch oficial (turnos 37 a 40)

1. Objetivo. Garantir que `cursor/yolov8-video-analysis-5e9e` contivesse tudo o que foi desenvolvido, sem regressões, sem código duplicado e sem branches pendentes.

2. Como funcionava antes. As funcionalidades estavam espalhadas em sete branches que divergiam a partir de `945ba0d`: - linha de times: `cursor/team-classifier-stability-5e9e`, `cursor/team-nomenclature-5e9e`, `cursor/person-class-lock-5e9e`; - linha de bola e passes: `cursor/possession-gap-passes-5e9e`, `cursor/ball-recall-near-weak-5e9e`, `cursor/player-ball-fallback-5e9e`, `cursor/possession-pass-score-5e9e`.

A branch oficial estava em `c7c8c6f`.

3. Qual era o problema. - A interface mostrava "Time Azul" porque a renomeação estava em outra branch. - `c7c8c6f` era um *squash* com pai único (`69e1d19`), embora o código fosse o de `c7d644a`. Um `git merge` das pontas trataria o refinamento de passes como edição dos dois lados em `run_yolo_analysis.py`, com 13 a 19 marcadores de conflito por merge.

4. Solução adotada. - **Auditoria (turno 38):** tabela por funcionalidade, com commit, branch, presença e conflitos. - **Plano (turno 39): cherry-pick na ordem de criação**, em vez de merge. - **Aprovação (turno 40):** com duas mudanças do usuário: - não trazer `2732558` e `e161b65` (versão intermediária do detector); - depois de cada cherry-pick, compilar, rodar os testes e o clipe de 150 frames, e confirmar ausência de regressão.

5. Implementação. Cherry-picks, na ordem:

Ordem	Commit novo	Original	Funcionalidade
1	`1fb0cef`	`945ba0d`	Âncora de cor e lock de time
2	`78a24f2`	`0a21239`	Bola fraca só continua a trilha
3	`a503632`	`4f81770`	Bola fraca próxima preferida à forte distante
4	`d06e77a`	`b3bd8e0`	Fallback da classe `ball`
5	`b47da34`	`ea7408a`	Passe por placar de posse
6	`effbbde`	`9a29e53`	Team 0 / Team 1
7	`e88f3ec`	`646c698`	PersonClassLock

Nos conflitos, ficou o detector por placar com os nomes Team 0 e Team 1. A trava de classe ficou depois da separação da bola do modelo de jogadores.

6. Fluxo. Para cada commit: cherry-pick ↓ resolve conflitos preservando a versão mais nova ↓ `py_compile` ↓ testes da etapa ↓ clipe de 150 frames (`--ball-detector yolo`) ↓ compara as contagens ↓ próxima etapa.

7. Problemas encontrados. O conflito real ficou em `pass_detector.py` e `test-pass-detector.py` (placar × renomeação) e em `run_yolo_analysis.py` (fallback × lock de classe).

8. Por que cherry-pick. Preserva a ordem histórica, evita reintroduzir o detector intermediário e permite testar cada passo isoladamente.

9. Impacto. A branch oficial passou a ser a única fonte de verdade.

10. Resultado. Branch em `e88f3ec`, com contagens idênticas em todas as etapas: - 150 quadros, 150 bolas, 2072 jogadores; - posse em 144 quadros, homografia válida nos 150; - times 888 e 1184, 14 trilhas travadas, 0 trocas depois da trava; - 0 passes.

Os seis testes unitários passaram no fim: times, recall da bola, fallback, passes, posse e trava de classe.

3.31 Depuração dos passes no vídeo de 401 frames (turnos 41 a 44)

1. Objetivo. Descobrir por que o painel mostrava zero passes depois da integração, e separar o que o detector realmente contava do que a interface exibia.

2. Como funcionava antes. O `PassDetector` por placar (`b47da34`) já rodava no worker. O painel "Análise da Partida" lia os eventos por `getEvents` em `public/js/analysisViewer.js`. No clipe de 150 frames a integração tinha registrado 0 passes (Seção 3.30), o que era coerente: não havia troca de possuidor.

3. Qual era o problema. - `getEvents` devolvia `blue_passes`, `red_passes` e `turnovers` fixos em 0. O painel não lia o detector. - No turno 43 o vídeo de teste não estava mais na máquina, então não dava para reprocessar. - No turno 44 o vídeo de 401 frames ("Gravação de Tela 2026-09-21 180218", 30 fps) foi reprocessado. O detector contou 3 passes e descartou 2 por tempo.

4. Solução adotada. Nenhuma alteração de código nesta etapa. A saída foi o diagnóstico quadro a quadro do `PassDetector` sobre o vídeo real, para decidir se o defeito estava no detector ou na interface.

5. Implementação. Não houve commit. A leitura usou as métricas já expostas pelo detector: `events`, `rejected_events` (motivo `timeout`) e conduções.

6. Fluxo observado no vídeo de 401 frames. - Passe contado: trilha 15 → 11, quadro 60, placar 100. - Passe contado: trilha 47 → 61, quadro 334, placar 80. - Passe contado: trilha 61 → 74, quadro 368, placar 100. - Descarte por timeout: trilha 11 → 41, quadros 178–227, intervalo de posse 1,633 s, distância 2303 cm (23,0 m). - Descarte por timeout: trilha 41 → 47, quadros 261–294, intervalo de posse 1,100 s, distância 2799 cm (28,0 m). - Terceiro candidato: a trilha 11 perde a posse no quadro 96 e a mesma trilha 11 a recupera no quadro 164. Isso é condução, não passe. - No quadro 392 a trilha 74 perde a posse e o vídeo acaba no quadro 401, sem receptor.

7. Problemas encontrados. Os dois descartes por timeout usavam o intervalo em que a posse ficou vazia (`PASS_MAX_GAP_SECONDS = 1,0`), não o tempo em que a bola estava no ar. Um passe de 23 m a uma velocidade realista ultrapassa 1 s de posse vazia e era morto antes do placar.

8. Por que essa leitura. Sem separar "o detector contou" de "o painel mostra", qualquer ajuste no placar seria feito no lugar errado. O painel continuava em zero mesmo com passes de placar 100.

9. Impacto. Nenhum módulo foi alterado. O diagnóstico motivou a Seção 3.32 (tempo de voo) e a Seção 3.34 (painel ao vivo).

10. Resultado. Três passes reais no detector, dois descartes por timeout e uma condução. O painel permanecia em zero por causa de `getEvents`.

3.32 Tempo de voo no lugar do intervalo de posse (turno 45, commit `78886a4`)

1. Objetivo. Julgar o tempo do passe pelo voo observado da bola, com um orçamento que cresce com a distância, em vez de cortar tudo que ficasse mais de 1 s sem possuidor.

2. Como funcionava antes. `_prepare_pass` comparava o intervalo de posse vazia (`gap_seconds`) com `PASS_MAX_GAP_SECONDS = 1,0`. Acima disso o evento ia para `discarded_timeout` e o placar nem era calculado. O histórico de trajetória tinha cerca de 32 quadros.

3. Qual era o problema. No vídeo de 401 frames: - 11 → 41 tinha sido marcado como trajetória "incompatible" só porque a janela de 32 quadros não cobria o voo. A premissa estava errada. - 41 → 47 tinha distância medida entre quadros diferentes (28,0 m). Medida dentro de um único quadro, a distância real era 15,9 m, e a bola chegou ao receptor em 0,63 s.

4. Solução adotada. - Um `flight_log` de 8 s (`PASS_FLIGHT_LOG_SECONDS = 8,0`) guarda, por quadro, a bola **observada** e a posição de cada jogador. Posições `predicted` e `remembered` não entram (`_is_observed`). - **Soltura:** último quadro em que A está dentro do raio de controle da bola observada (`CONTROL_RADIUS_CM = 250` ou `CONTROL_RADIUS_PX = 90`). - **Chegada:** primeiro quadro posterior em que B está dentro desse raio. - O tempo usado (`timing_seconds`) é o mínimo entre o intervalo de posse e o voo, desde que o atraso de recepção (`gained_frame − arrival_frame`) seja de no máximo `PASS_MAX_RECEPTION_LAG_SECONDS = 1,0`. - Orçamento: `max_flight_seconds(d) = clamp(d / 800 + 0,5, 1, 4)` segundos. 800 cm/s é `PASS_MIN_BALL_SPEED_CM_S`. O teto é `PASS_MAX_FLIGHT_SECONDS = 4,0`. Sem distância, o orçamento volta a 1,0 s. - Faixa **base** se `timing_seconds ≤ 1,0`; senão faixa **estendida**. - A faixa estendida perde o bônus de intervalo curto (10) e leva `unverified_extension = −20`, a menos que a trajetória seja `compatible` **e** haja pelo menos `PASS_EXTENDED_MIN_FLIGHT_SAMPLES = 2` amostras de bola observada estritamente entre soltura e chegada. - Velocidade implícita acima de `PASS_MAX_BALL_SPEED_CM_S = 4000` (40 m/s) vira `discarded_implausible_speed`. A velocidade usa o intervalo mais generoso: distância dividida por `(gained_frame − release_frame) / fps`. - Adversário sozinho na bola por pelo menos `max(3 quadros, 0,25 s)` entre soltura e chegada vira `discarded_intercepted`. - A distância A–B é lida **dentro do mesmo quadro** (`same_frame`), no quadro de soltura ou no de chegada, para o jitter da homografia se cancelar. Só se isso falhar usa-se `cross_frame`.

5. Implementação. Commit `78886a4` (12/10/2026), mensagem "Judge pass timing on observed ball flight with a distance-scaled budget". Arquivo `pass_detector.py`: `flight_log`, `_log_frame`, `_flight_evidence`, `_pass_distance`, `_implied_speed`, `_interceptor`, `max_flight_seconds`, constantes novas no topo do módulo. `scripts/test-pass-detector.py` ganhou 7 casos; os fixtures passaram a 10 fps. Total de casos do arquivo: 23.

6. Fluxo. `update` ↓ `_log_frame` (só bola observada, janela de 8 s) ↓ abertura do pending quando a posse esvazia ↓ `_resolve` quando outro possuidor confirma ↓ `_prepare_pass` calcula soltura, chegada, `timing_seconds`, faixa, distância no mesmo quadro e trajetória ↓ cortes físicos na ordem timeout, curto (< 150 cm), longo (> 7000 cm), velocidade > 40 m/s, interceptação ↓ placar ↓ `_record_prepared`.

7. Problemas encontrados. - A trajetória "incompatible" de 11 → 41 era artefato da janela curta, não da bola. - A distância entre quadros diferentes inflava 41 → 47 de 15,9 m para 28,0 m. - O custo da etapa subiu de 0,039 ms/quadro para 0,072 ms/quadro. - O painel continuava em zero. O commit não tocou o frontend.

8. Por que essa solução. O intervalo sem possuidor mistura confirmação da máquina de estados (5 quadros para ganhar, 8 para perder) com o voo da bola. O voo observado separa as duas coisas. O orçamento `d/8 m/s + 0,5 s` admite um passe longo lento e ainda corta uma bola solta. Medir A e B no mesmo quadro evita somar o salto da homografia.

A robustez foi checada variando a velocidade mínima de 6 a 12 m/s, a máxima de 30 a 50 m/s, a folga de recepção de 0,25 a 1 s, e descartando de 10% a 30% dos quadros com bola. A contagem dos cinco passes se manteve.

9. Impacto. Isolado em `pass_detector.py` e no teste. `PossessionEngine`, detectores, ByteTrack, homografia e frontend ficaram como estavam.

10. Resultado. No vídeo de 401 frames a contagem foi de 3 para 5 passes. - 11 → 41: voo de 1,63 s, faixa estendida, trajetória compatível, placar 90. - 41 → 47: a bola chegou em 0,63 s, faixa base, placar 100. - Os outros três (15 → 11, 47 → 61, 61 → 74) permaneceram.

3.33 Passe longo com bola oculta e o painel ainda em zero (turnos 46 a 49)

1. Objetivo. Entender o que acontece quando a bola some no meio de um passe longo, e explicar por que um passe de placar 100 não aparecia na interface.

2. Como funcionava antes. O voo da Seção 3.32 já estava no código. A BallMemory dura `BALL_MEMORY_MAX_MISSING_FRAMES = 5` quadros. O `flight_log` ignora bola `predicted` e `remembered`. `in_flight_samples` conta só quadros estritamente entre soltura e chegada. A faixa estendida sem trajetória compatível e sem pelo menos 2 amostras leva −20.

3. Qual era o problema. Uma simulação com `PossessionEngine` + `PassDetector` a 30 fps, passe de 30 m, mostrou: - oclusão de 1,20 s: placar 100, porque a memória mantém a posse e o intervalo de posse fica ≤ 1 s (faixa base); - oclusão de 1,33 s, 2 s e 3 s: placar 70 (40 + 30 + 0 + 10 + 10 − 20), abaixo do limiar 80, então o passe vai para `unconfirmed_events`; - oclusão de 5 s: timeout.

4. Solução adotada. Nenhuma implementação. A correção discutida, e **não feita**, foi retirar o −20 quando as duas pontas estão observadas, a trajetória é compatível e a velocidade fica entre 8 e 40 m/s. A BallMemory de 5 quadros continuaria curta demais para preencher o meio do voo, mas o placar deixaria de punir a ausência de amostras no meio se as pontas e a direção bastassem.

No turno 48 o passe 15 → 11 foi rastreado linha a linha: ele **foi registrado** no quadro 60 com placar 100 (40 + 30 + 10 + 10 + 10 + 0). O painel mostrava 0 porque `getEvents` fixava os contadores em zero. No turno 49 ficou claro que os passes eram contados ao vivo no Python e não iam no payload de progresso, nem eram lidos pelo painel, inclusive em 100%.

5. Implementação. Não houve commit.

6. Fluxo da simulação. Posse confirmada em A ↓ bola some ↓ memória segura a posse por 5 quadros ↓ depois a posse esvazia ↓ bola reaparece em B ↓ B confirma em 5 quadros ↓ `_prepare_pass` mede o voo ↓ se a faixa for estendida e `in_flight_samples < 2`, aplica −20 ↓ placar 70 ↓ `unconfirmed`.

7. Problemas encontrados. A memória de posse e o log de voo usam fontes diferentes. A memória aceita bola prevista para não perder o possuidor; o log de voo recusa essa mesma bola para não inventar trajetória. As duas escolhas são coerentes sozinhas e, juntas, punem o passe longo ocluído.

8. Por que não implementar agora. A regra proposta mexe no placar de todo passe estendido. Sem um vídeo em que o caso apareça de verdade, a mudança ficaria sem medição. O defeito visível para o usuário era o painel, e esse sim tinha causa fechada.

9. Impacto. Nenhum. A proposta permanece na Seção 5.

10. Resultado. O passe de placar 100 existia no worker. O painel não tinha como mostrá-lo.

3.34 Painel de passes ao vivo (turno 50, commit `3d97765`)

1. Objetivo. Mostrar, durante a análise, os passes que o `PassDetector` já tinha contado, agrupados por time.

2. Como funcionava antes. `getEvents` em `analysisViewer.js` devolvia passes e trocas de posse fixos em 0. O payload de progresso do worker trazia o snapshot de posse em `events`, mas não a lista de passes.

3. Qual era o problema. Passes com placar 100 (15 → 11 no quadro 60, por exemplo) nunca chegavam ao cartão. O usuário via zero do primeiro quadro ao último.

4. Solução adotada. Copiar `passes.total_passes` e `passes.events` do JSON que o worker já emitia, guardar isso em `progress_stats.passes` e, no frontend, contar os eventos cujo `team` é exatamente `"Team 0"` ou `"Team 1"`.

5. Implementação. Commit `3d97765` (12/10/2026), "Show backend pass counts on the match panel". Arquivos: - `run_yolo_analysis.py`: o payload de progresso passa a incluir `passes`. - `server/services/analysis/yolo/passProgress.js`: `passProgressFromPayload` devolve `{ total_passes, events }` (as trocas de posse entraram depois, na Seção 3.42). - `YoloAnalysisProvider.js` e `AnalysisService.js`: gravam o objeto em `progress_stats` e no resultado. - `public/js/passCounts.js`: `countPassesByTeam`. - `public/js/analysisViewer.js`: `backendPasses` / `getEvents` leem o objeto em vez de devolver zero. - `scripts/test-pass-panel.mjs`.

`pass_detector.py` não foi alterado.

6. Fluxo. Worker emite `passes` junto com o progresso (a cada 5% , frame de debug ou mudança de posse) ↓ Node copia `total_passes` e `events` para `progress_stats` ↓ o frontend faz polling a cada 200 ms ↓ `countPassesByTeam` soma eventos com `team === "Team 0"` e `team === "Team 1"` ↓ o cartão atualiza.

7. Problemas encontrados. Um evento sem `team` exatamente igual a essas duas strings não entra no cartão. Isso explica, mais adiante, parte da divergência 2 contra 5 (Seção 3.36): se o JSON salvo tivesse outro nome de time, o cartão contaria menos do que o worker.

8. Por que essa solução. O detector já era a fonte da verdade. Duplicar a contagem no JavaScript criaria um segundo placar. O frontend só agrupa.

9. Impacto. Worker (campo novo no payload), Node (cópia do campo) e frontend (leitura). A lógica de passe, posse, bola, times e homografia não mudou.

10. Resultado. No vídeo de teste, ao final, Team 0 = 0 e Team 1 = 5. No meio da análise, logo depois de 15 → 11, Team 1 mostra 1.

3.35 Conflitos com `main` que não foram commitados (turnos 51 a 59)

1. Objetivo. Trazer `origin/main` (`7f4dd14`, busca híbrida da bola) para a branch oficial sem perder o detector de passes, a nomenclatura Team 0 / Team 1 nem o recall da bola.

2. Como funcionava antes. A branch oficial estava em `3d97765`. `main` tinha evoluído a busca da bola (crop, semi-global, full global) com limiar de confiança 0,25 e sem o fallback da classe `ball` do modelo de jogadores. A branch oficial usava `keep_recoverable_ball_candidates`, confiança YOLO 0,02, `select_player_ball_fallback` e `weak_yolo_continuation`, e emitia times `"Team 0"` / `"Team 1"`. `main` emitia `"blue"` / `"red"`, com o painel em "Time Azul" / "Time Vermelho".

3. Qual era o problema. Dois conflitos não têm resolução automática: - **Nomes de time.** Arquivos: `public/app.html`, `analysisViewer.js` (`possessionLabel`), `matchEvents.js`, `possession.py` (`TEAM_NAME_BY_ID`) e `scripts/test-possession-engine.py`. Fundir para `"blue"`/`"red"` faria o painel de passes (que compara com `"Team 0"`/`"Team 1"`) zerar. - **Política da bola** em `run_yolo_analysis.py`. Ficar com a lista crua e o limiar 0,25 de `main` apaga o recall. Apagar a atribuição de `weak_yolo_continuation` e deixar a leitura gera `NameError`.

Conflitos simples (import de `passProgress` em `AnalysisService`, `byId` no viewer, contagens de passe, ligação do `PassDetector` e do `PersonClassLock`, constantes de recall) foram resolvidos na árvore de trabalho e **não foram commitados**.

4. Solução adotada. Nenhuma. O merge ficou aberto de propósito. O aviso do GitHub continua porque o remoto ainda está em `3d97765`. Cada "Fix with agent" para no mesmo par de conflitos. No turno 59 ficou registrado que o trabalho na branch pode continuar: o aviso só impede o merge em `main`.

5. Implementação. Não houve commit de merge.

6. Fluxo. `git merge origin/main` ↓ conflitos simples resolvidos na árvore ↓ conflitos de nome de time e de política da bola deixados em aberto ↓ merge não commitado.

7. Problemas encontrados. Resolver "aceitando o nosso" ou "aceitando o deles" num arquivo só reintroduz o outro defeito. Os dois lados são intencionais.

8. Por que deixar aberto. Escolher a nomenclatura e a política da bola é uma decisão do projeto, não um conflito de texto. Commitar um dos lados sem essa escolha quebraria ou o painel ou o recall.

9. Impacto. A branch oficial no remoto não contém `main`. Quem clonar `cursor/yolov8-video-analysis-5e9e` em `3d97765` ou posterior **não** tem o merge. O aviso do GitHub permanece.

10. Resultado. Merge não integrado. Os dois conflitos estão listados na Seção 5.

3.36 Divergência de 2 passes na tela contra 5 nesta sessão (turnos 53, 60 e 61)

1. Objetivo. Explicar por que a tela do usuário mostrava 2 passes e a execução desta sessão, no mesmo vídeo, contava 5.

2. Como funcionava antes. O painel soma eventos com `team` exatamente `"Team 0"` ou `"Team 1"` (`countPassesByTeam`). O worker, nesta sessão, emitiu cinco eventos, todos Team 1: - 15 → 11, quadro 60, confiança 100; - 11 → 41, quadro 227, confiança 90; - 41 → 47, quadro 294, confiança 100; - 47 → 61, quadro 334, confiança 90; - 61 → 74, quadro 368, confiança 100.

3. Qual era o problema. A tela do usuário mostrava 2. Não há, no histórico, uma segunda execução com log que reproduza o 2. Três lugares guardam a conta, e eles não são equivalentes: - `analyses.result_json`, coluna no SQLite `conversor-video/data/app.db`, chave `passes` — é o resultado salvo, com a lista de eventos; - `GET /api/analyses/:id`, campo `analysis.result_json.passes` — a mesma coisa, servida pela API; - `progress_stats.passes` — só `total_passes`, `events` e, depois, `turnovers`; mora na memória e some se o processo reinicia; - `conversor-video/data/analyses/<user_id>/<analysis_id>/annotated.mp4.profile.json`, chave `pass_detection` — só contadores, sem a lista de eventos.

`discarded_direction_check` existe como contador, mas o detector atual não o incrementa. O motivo por candidato fica em `rejected_events[].reason` (`timeout`, `short`, `long`, `implausible_speed`, `intercepted`). Placar abaixo de 80 vai para `unconfirmed_events`, não para o cartão.

4. Solução adotada. Nenhuma alteração. Foi entregue um script Python só de leitura, com `sqlite3`, para rodar dentro de `conversor-video` e filtrar análises cujo nome de vídeo contém "180218" ou "Grava". A saída nunca voltou.

5. Implementação. Não houve commit. Também não foram escritos testes que afirmem "este vídeo tem 5 passes": a contagem depende do modelo, dos pesos e do vídeo, e o ambiente desta sessão não tem `torch` nem os pesos.

6. Fluxo de leitura proposto. Abrir `data/app.db` ↓ filtrar pelo nome do vídeo ↓ imprimir `result_json.passes.total_passes`, cada evento (`from_track_id`, `to_track_id`, `team`, `frame`, `confidence`) e os contadores de descarte ↓ comparar com o 2 da tela.

7. Problemas encontrados. Sem o JSON salvo na máquina do usuário, as hipóteses (análise antiga, anterior ao voo; `progress_stats` de uma execução interrompida; time com outro nome) ficam abertas.

8. Por que não tratar como bug geral. O caminho de leitura do painel é uma comparação de string com o que o worker gravou. Se o banco disser 5 e a tela disser 2 na mesma análise, aí sim o defeito está na leitura. Se o banco disser 2, a análise é de uma versão anterior do detector.

9. Impacto. Nenhum módulo. A divergência permanece na Seção 5.

10. Resultado. Inconclusivo. A execução documentada aqui tem 5 passes, todos Team 1. A execução que mostrou 2 não foi inspecionada.

3.37 Ordem sugerida das próximas estatísticas (turno 62)

1. Objetivo. Escolher a próxima estatística que o painel pode mostrar com o que o pipeline já produz, antes de 11 de novembro.

2. Como funcionava antes. O painel mostrava passes por time. Posse por time, trocas de posse e mapa de calor da bola não existiam como cartões. Trocas de posse já eram contadas no detector e o cartão estava em zero.

3. Qual era o problema. Distância e velocidade por jogador somam saltos de homografia e quebras de trilha. Mapa de calor de jogadores mistura 22 trilhas, goleiro e árbitro. "Condução" no detector é o retorno do mesmo `track_id`, não um drible. Recuperações repetem a troca de posse.

4. Solução adotada. Nenhuma implementação. A ordem recomendada foi: posse por time (com o tempo sem possuidor), depois trocas de posse, depois mapa de calor da bola.

5. Implementação. Não houve commit. As três recomendações viraram as Seções 3.38, 3.42 e 3.44.

6. Fluxo. Não se aplica.

7. Problemas encontrados. Não se aplica.

8. Por que essa ordem. As três leem um estado que o quadro já tem (time do possuidor, contador de turnover, ponto de campo da bola observada). Nenhuma exige uma trilha estável por muitos segundos.

9. Impacto. Nenhum nesta etapa. A decisão delimitou o que entrou no código até `bf0cc80`.

10. Resultado. Três estatísticas escolhidas e três deixadas de fora (distância/velocidade, mapa de jogadores, recuperações).

3.38 Posse por time (turno 63, commit `aae84a1`)

1. Objetivo. Mostrar, ao vivo e no resultado final, a fração dos quadros em que cada time era o possuidor confirmado, e a fração sem possuidor.

2. Como funcionava antes. O `PossessionEngine` já publicava `current_possession_team` por quadro. O painel mostrava só o rótulo do possuidor atual. Não havia acumulador.

3. Qual era o problema. Um rótulo "Team 1" no último quadro não diz quanto tempo aquele time ficou com a bola. O último quadro do vídeo de teste está vazio, então o rótulo final ficava "Aguardando análise" mesmo com o time tendo ficado com a bola na maior parte do lance.

4. Solução adotada. Um objeto novo, `possession_by_team`, ao lado de `passes`. A cada quadro o acumulador soma o time que o `PossessionEngine` **já confirmou**. Qualquer outro valor (vazio, time desconhecido) incrementa `frames_without_possessor`. O percentual é sobre **todos** os quadros, e os três percentuais somam 100. Segundos = quadros / fps.

5. Implementação. Commit `aae84a1` (18/10/2026), "Show possession share by team during analysis". - `possession_stats.py` (novo): classe `PossessionTeamAccumulator`. `fps` inválido cai para 30. `observe(team, pass_events)` incrementa um dos três contadores. `as_dict` calcula percentuais com uma casa (`round(100 * frames / frames_total, 1)`) e segundos com três casas (`round(frames / fps, 3)`). - `run_yolo_analysis.py`: chama `possession_team_stats.observe(team, pass_events)` depois do `PassDetector` e coloca `possession_by_team` no payload e no resultado. - `possessionProgress.js` (novo): valida e copia o objeto. - `YoloAnalysisProvider.js`, `AnalysisService.js`: guardam em `progress_stats` e em `result_json`. - `public/js/possessionCounts.js` (novo): `readPossessionByTeam`, `formatPossessionPercent`, `formatPossessionSeconds`, `formatPossessionDetail`. - `analysisViewer.js`, `app.html`, `style.css`: cartão "Posse por Time". - Testes: `scripts/test-possession-stats.py` e `scripts/test-possession-panel.mjs`.

O `PassDetector` não decide a posse. O acumulador não abre nem troca possuidor.

6. Fluxo. `PossessionEngine.update` ↓ snapshot com time ou vazio ↓ `PassDetector.update` ↓ `observe(team)` ↓ se o time é `"Team 0"` ou `"Team 1"`, soma o quadro naquele time e zera o bloco aberto; senão soma em sem possuidor ↓ `as_dict` ↓ payload ↓ painel.

7. Problemas encontrados. No vídeo de 401 frames, antes da reatribuição (Seção 3.39), a maior parte dos quadros ficava em "sem posse" porque o passe ainda não tinha sido confirmado: - quadro 12: 100% sem possuidor; - quadro 90: 50% / 50%; - quadro 250: Team 1 35,2%, sem possuidor 64,8%; - final: Team 1 43,4% (174 quadros, 5,8 s), sem possuidor 56,6% (227 quadros, 7,6 s).

O rótulo ao vivo do último quadro continua "Aguardando análise" quando aquele quadro não tem possuidor. Isso é o snapshot, não o acumulador.

8. Por que essa solução. Recontar a posse no frontend duplicaria a máquina de estados. O percentual sobre todos os quadros mantém os três números comparáveis e evita esconder o tempo em que ninguém estava confirmado.

9. Impacto. Worker (acumulador e campo novo), Node (cópia) e frontend (cartão). Bola, times, homografia, RF-DETR e a lógica de passe não mudaram.

10. Resultado. Cartão de posse por time ao vivo. No vídeo de teste, Team 0 ficou em 0% — todos os possuidores confirmados eram Team 1. Mais da metade dos quadros ainda aparecia sem possuidor, o que a Seção 3.39 trata.

3.39 Reatribuição do intervalo quando o passe do mesmo time confirma (turnos 65 a 68, commit `78940e3`)

1. Objetivo. Quando um passe do mesmo time é confirmado, contar os quadros do intervalo (a bola no ar, sem possuidor) para aquele time.

2. Como funcionava antes. Cada quadro sem possuidor ia para `frames_without_possessor` e lá ficava. O passe, ao confirmar, passava a contar só dali para a frente. No final do vídeo de teste, 56,6% dos quadros apareciam sem dono, embora a maior parte fosse o voo de um passe do Team 1.

3. Qual era o problema. O intervalo entre soltura e confirmação é exatamente o trecho que o usuário entende como "o time ainda está com a bola". Deixá-lo em "sem posse" subestimava o time que completou o passe.

4. Solução adotada. O acumulador guarda um único bloco aberto `{team, frames}` — o time do último possuidor e a quantidade de quadros sem possuidor desde então. Não guarda a lista de quadros. Quando `passes.events` ganha um passe **novo** e **todos** os eventos novos são do mesmo time, e esse time é o do bloco aberto, aqueles quadros saem de "sem posse" e entram no time. O quadro da confirmação já foi somado no time e não é somado de novo. `_seen_pass_events` impede contar o mesmo evento duas vezes. `_take_new_pass_team` devolve `None` se os eventos novos misturam times ou não são do tipo `"pass"`.

O que **não** move o bloco: condução, troca de posse, disputa, passe não confirmado (placar < 80), ou o vídeo acabar com o bloco aberto.

Uma alternativa mais estrita foi discutida e **não escolhida**: mover só os quadros entre `release_frame` e `arrival_frame`. A versão adotada move o bloco inteiro, porque o bloco já é "desde o último possuidor deste time".

5. Implementação. Commit `78940e3` (18/10/2026), "Reassign an open possession gap when the same team completes a pass". Arquivos: `possession_stats.py` (`_open_team`, `_open_frames`, `_seen_pass_events`, `_take_new_pass_team`, `_reassign`), `possessionProgress.js`, `run_yolo_analysis.py` (o `observe` passa a receber `pass_events`), `test-possession-stats.py`, `test-possession-panel.mjs`. Contadores de auditoria `frames_reassigned_team_0` e `frames_reassigned_team_1` entram no dict e **não** entram no percentual. `pass_detector.py` não foi alterado. `_reassign` limita a transferência a `frames_without_possessor`, para o contador não ficar negativo.

6. Fluxo. Quadro sem possuidor e último time conhecido ↓ abre ou incrementa o bloco ↓ quadro com possuidor ↓ lê os eventos de passe novos ↓ se o time único desses eventos é o do bloco, subtrai os quadros de "sem posse" e soma no time, e incrementa `frames_reassigned_*` ↓ zera o bloco ↓ segue.

Durante o voo o rótulo ao vivo continua "Aguardando análise". A barra corrige para trás no quadro em que o passe confirma.

7. Problemas encontrados. O bloco é um único par `{time, quadros}`. Dois intervalos seguidos do mesmo time se fundem, o que neste vídeo é o comportamento desejado. Um passe de outro time no meio dos eventos novos cancela a reatribuição daquele quadro (`any(team != teams[0])`).

8. Por que essa solução. O passe confirmado do mesmo time é a evidência de que o intervalo pertencia àquele time. Guardar só a contagem evita uma segunda linha do tempo. Amarrar a reatribuição ao evento novo evita mover o bloco num quadro em que nada mudou.

9. Impacto. Só o acumulador de posse e a cópia do payload. O detector de passes não mudou. O cartão lê os mesmos percentuais de antes; os números é que passam a incluir o voo.

10. Resultado. No vídeo de 401 frames, depois da reatribuição: - quadro 90: Team 1 81,1% (os 18,9% restantes são o trecho anterior à primeira posse); - quadro 250: Team 1 66,0%, sem possuidor 34,0% (antes, 64,8% sem possuidor); - final: Team 1 76,3% (306 quadros, 10,2 s), sem possuidor 23,7% (95 quadros, 3,2 s).

Os 23,7% que sobram são o começo do vídeo, o retorno do mesmo jogador (condução, que não reatribui) e o trecho final.

3.40 Por que 5 passes e não 7 (turnos 69 a 73)

1. Objetivo. Explicar os dois lances do vídeo de 401 frames que parecem passe e não entram na contagem.

2. Como funcionava antes. Com o voo (Seção 3.32) e a reatribuição (Seção 3.39), o detector contava 5 passes, todos Team 1.

3. Qual era o problema. A linha do tempo de posse do vídeo é:

Quadros	Possuidor	O que o detector faz
18–31	trilha 15	primeira posse
32–59	vazio	intervalo
60–95	trilha 11	passe 15 → 11 no quadro 60
96–163	vazio	a bola vai e volta
164–177	trilha 11 de novo	condução, não passe
178–226	vazio	intervalo
227–260	trilha 41	passe 11 → 41
261–293	vazio	intervalo
294–328	trilha 47	passe 41 → 47
329–333	vazio	intervalo
334–350	trilha 61	passe 47 → 61
351–367	vazio	intervalo
368–391	trilha 74	passe 61 → 74
392–401	vazio	vídeo acaba, sem receptor

No turno 71 o vídeo foi revisto: - O segundo lance é uma tabela (um-dois). A bola sai de um jogador de camisa branca por volta de 3,0 s, um companheiro a redireciona por volta de 3,8 s (quadro ~114) e ela volta por volta de 4,5 s. O jogador da tabela não segura a bola pelos 5 quadros de confirmação, então nenhum possuidor novo é confirmado. Quando a trilha 11 recupera a posse no quadro 164, o detector registra condução. O teste que fixa isso é `test_same_player_return_is_a_carry`. - O último lance é um chute em direção ao gol, por volta de 07:38–07:39, sem receptor. O vídeo acaba no quadro 401.

4. Solução adotada. Nenhuma neste turno. A proposta, implementada na Seção 3.41, foi: só quando o lance **ia virar condução**, procurar um companheiro B que seja o jogador mais próximo da bola observada dentro do raio de disputa (1,2 m), com a bola se aproximando e depois saindo em direção a A, e sem adversário sozinho na bola. Os dois lances da tabela recebem o placar que já existe. A confirmação global de 5 quadros **não** é reduzida.

5. Implementação. Não houve commit neste turno.

6. Fluxo do falso "quinto passe que falta". A solta ↓ companheiro toca sem completar 5 quadros ↓ A recebe de volta ↓ `possessor_id == pending["from_id"]` ↓ condução ↓ o intervalo não é reatribuído.

7. Problemas encontrados. Baixar `POSSESSION_CONFIRM_FRAMES` de 5 para 1 faria o toque da tabela virar posse, mas também confirmaria marcação, bola ruim e trocas de um quadro no resto do vídeo. A proposta recusa esse caminho.

8. Por que olhar só a condução. O caso que falta é exatamente "o mesmo jogador recebe de volta". Os outros retornos continuam condução. O chute final não tem receptor e não entra nessa regra.

9. Impacto. Nenhum ainda. A decisão restringe a mudança ao `PassDetector`.

10. Resultado. Diagnóstico fechado: uma tabela contada como condução, e um chute sem receptor. Faltam dois passes, não um ajuste de limiar.

3.41 Tabela (passe de parede) contada como dois passes (turno 74, commit `0b217e0`)

1. Objetivo. Quando o mesmo jogador recebe a bola de volta depois de um toque de um companheiro, contar dois passes, sem baixar a confirmação de posse.

2. Como funcionava antes. `update`, ao ver `possessor_id == pending["from_id"]`, incrementava `conductions` e fechava o pending. O teste `test_same_player_return_is_a_carry` cobria esse caminho. A tabela do vídeo (quadros 96–164) caía nele.

3. Qual era o problema. O companheiro toca a bola dentro do raio de disputa e não permanece com ela pelos 5 quadros. A máquina de posse, corretamente, não cria um possuidor. O detector, por só olhar o possuidor confirmado, via "A devolveu para A".

4. Solução adotada. Antes de contar a condução, `_apply_wall` procura a parede: - o time de A é `"Team 0"` ou `"Team 1"` e o pending não está `contested`; - nenhum adversário fica sozinho na bola por `max(3 quadros, 0,25 s)` entre a soltura e o retorno (`_opponent_owns_ball`); - em cada quadro do `flight_log` com bola observada, entre a soltura e o retorno, o jogador mais próximo dentro do raio de disputa (`PASS_CONTEST_RADIUS_CM = 120`, ou 45 px) é um companheiro diferente de A; - se os dois times estão juntos na bola naquele quadro, a parede é rejeitada; - entre os companheiros, fica o que chegou mais perto; a posição usada é a desse quadro; - `_ball_approaches_and_leaves` exige aproximação até o toque, saída depois do toque e retorno em direção a A, cada um com pelo menos o movimento mínimo (`PASS_DIRECTION_MIN_MOTION_CM = 80` ou 20 px).

Os dois lances passam por `_prepare_pass` com `wall=True` e pelo mesmo placar, os mesmos cortes físicos e o mesmo limiar 80. Se **qualquer** lance cai em `_WALL_PHYSICAL_REJECTIONS = {timeout, short, long, implausible_speed, intercepted}`, a tabela inteira é descartada e o lance continua condução. Um lance abaixo de 80 não está nesse conjunto: ele vai para `unconfirmed_events` e o outro lance ainda pode ser contado. Os dois lances, se contados, incrementam `wall_passes`.

5. Implementação. Commit `0b217e0` (18/10/2026), "Count a one-touch wall as two passes when the same player receives the ball back". Arquivos: `pass_detector.py` (`_apply_wall`, `_find_wall`, `_release_frame`, `_opponent_owns_ball`, `_ball_approaches_and_leaves`, `_distance_series`) e `scripts/test-pass-detector.py` com 6 testes novos: tabela real gera 2 passes; recuperação solo continua condução; adversário na bola; companheiro fora do raio; toque de um quadro; lance abaixo do placar. `POSSESSION_CONFIRM_FRAMES` permanece 5. BallMemory, BallTracker, RF-DETR, TeamClassifier, homografia e frontend não foram alterados.

6. Fluxo. Posse volta para o mesmo `track_id` ↓ `_apply_wall` ↓ `_find_wall` ↓ dois `_prepare_pass` ↓ se um lance tem rejeição física, devolve falso e o `update` conta condução ↓ senão `_record_prepared` nos dois ↓ pending fecha e A volta a ser o último possuidor.

7. Problemas encontrados. A parede depende de bola **observada** no `flight_log`. Um toque em que a bola só existe como previsão não forma parede. O raio de 1,2 m é o de disputa do detector, menor que o raio de controle da posse (2,5 m): um companheiro a 2 m da bola não é parede.

8. Por que essa solução. Ela usa a evidência que a posse confirmada não tem (quem chegou perto da bola observada) e recusa mudar a confirmação de 5 quadros, que protege o resto do vídeo. Reusar `_prepare_pass` impede um segundo conjunto de regras para "passe curto de tabela".

9. Impacto. Isolado no `PassDetector`. A reatribuição de posse (Seção 3.39) passa a ver os dois eventos novos. Como são do mesmo time, o bloco aberto vai para esse time. Foi o que subiu a posse do Team 1 no trecho da tabela.

10. Resultado. No vídeo de 401 frames (turno 75): - quadro 90: inalterado, 1 passe, Team 1 81,1%; - quadro 164: a tabela fecha e os passes sobem de 1 para 3, posse 89,6%; - quadro 250: 4 passes, Team 1 93,2%, sem possuidor 6,8%; - final: 7 passes, Team 1 93,3% (374 quadros, 12,5 s), sem possuidor 6,7% (27 quadros, 0,9 s).

Os 0,9 s que sobram são o início do vídeo e o chute final.

3.42 Trocas de posse no painel (turnos 76 e 77, commit `14fb7a5`)

1. Objetivo. Mostrar no cartão o número de trocas de posse que o detector já calculava.

2. Como funcionava antes. Em `_resolve`, quando os dois possuidores confirmados têm times conhecidos e diferentes, o detector incrementa `turnovers` e anexa `turnover_events`. Disputa (`contested`, os dois times dentro do raio de disputa) **não** entra nesse contador: ela incrementa `disputes` e retorna antes. A confiança da troca é `TURNOVER_BASE_SCORE = 70`, mais 10 se a trajetória é compatível e menos 10 se é incompatível. O cartão `analysisTurnovers` estava fixo em 0, no mesmo ponto em que os passes estavam antes da Seção 3.34.

3. Qual era o problema. O número existia em `pass_metrics["turnovers"]` e não saía no objeto `passes` que o painel lê.

4. Solução adotada. Copiar o inteiro `turnovers` junto com `total_passes` e `events`. Sem recalcular no JavaScript e sem mudar a regra (times diferentes, posse confirmada, disputa fora).

5. Implementação. Commit `14fb7a5` (19/10/2026), "Show the detector's turnover count on the match panel". - `run_yolo_analysis.py`: os dois payloads de `passes` (progresso e resultado) incluem `turnovers`. - `passProgress.js`: copia `turnovers` com `Math.round`, e trata valor não finito ou ≤ 0 como 0. - `analysisViewer.js`: `readTurnovers` / `getEvents` leem o campo. - `scripts/test-pass-panel.mjs`: começa em zero, sobe quando o outro time confirma, disputa permanece 0, o valor final persiste.

`pass_detector.py` não foi alterado. `POSSESSION_CONFIRM_FRAMES` permanece 5.

6. Fluxo. Possuidor novo de outro time ↓ `_resolve` vê times diferentes ↓ `turnovers += 1` ↓ payload `passes.turnovers` ↓ `passProgressFromPayload` ↓ cartão.

7. Problemas encontrados. O cartão mostra o contador, não a lista `turnover_events`. Um valor ausente no payload vira 0, para uma análise antiga não quebrar o cartão.

8. Por que essa solução. A troca já estava definida no detector desde o placar (Seção 3.29). Mostrá-la é o mesmo tipo de cópia da Seção 3.34. Recontar "troca" no frontend a partir dos eventos de passe perderia exatamente os lances que não são passe.

9. Impacto. Payload, ponte Node e cartão. A classificação passe / troca / disputa / condução não mudou.

10. Resultado. O cartão deixa de nascer em zero fixo e acompanha `pass_metrics["turnovers"]`. No vídeo de 401 frames os possuidores confirmados são todos Team 1, então a troca de posse desse vídeo é 0. Disputa continua fora do número.

3.43 Opinião sobre a estatística seguinte (turno 78)

1. Objetivo. Escolher o que implementar depois das trocas de posse, com o que o quadro já produz.

2. Como funcionava antes. O painel tinha passes por time, posse por time (com reatribuição) e trocas de posse. Não tinha mapa espacial da bola.

3. Qual era o problema. As candidatas restantes tinham defeitos já medidos ou discutidos: - distância e velocidade por jogador somam saltos de homografia e quebras de `track_id`; a velocidade máxima é o pior caso; - mapa de calor de jogadores espalha 22 trilhas, mais goleiro e árbitro, pelo campo; - recuperações repetem a troca de posse; - chutes, gols e rede de passes exigem uma classe de chute que não existe. O chute final do vídeo de teste foi excluído exatamente por não ter receptor.

4. Solução adotada. Nenhuma implementação neste turno. A ordem recomendada foi: 1. mapa de calor da bola, só com bola observada; 2. posse por faixa do campo (0–40 m, 40–80 m, 80–120 m), com rótulos esquerda / meio / direita — não ataque / defesa, porque a direção de ataque é desconhecida e inverte no intervalo; 3. duração média da posse.

5. Implementação. Não houve commit. O item 1 virou a Seção 3.44. Os itens 2 e 3 estão na Seção 5.

6. Fluxo. Não se aplica.

7. Problemas encontrados. Não se aplica.

8. Por que o mapa da bola primeiro. Cada quadro já tem `field_x` / `field_y` da bola, ou a ausência deles. A conta é um incremento numa grade. Não depende de trilha de jogador nem de uma classe nova.

9. Impacto. A decisão deixou distância, velocidade, mapa de jogadores, recuperações, chutes e rede de passes fora do código até `bf0cc80`.

10. Resultado. Mapa de calor da bola escolhido como próxima implementação.

3.44 Mapa de calor da bola observada (turnos 79 a 81, commit `bf0cc80`)

1. Objetivo. Acumular, ao vivo, em que zona do campo a bola observada esteve, e desenhar isso no painel.

2. Como funcionava antes. A homografia já gravava `field_x` e `field_y` em centímetros num campo de 12000 × 7000 (`SoccerPitchConfiguration`). Nada acumulava esses pontos. O painel não tinha área de mapa.

3. Qual era o problema. Os passes e a posse dizem quem teve a bola, não onde ela esteve. Contar bola prevista ou lembrada pintaria o campo nos quadros em que o detector não viu a bola.

4. Solução adotada. `BallHeatmapAccumulator` com grade fixa de 12 colunas por 8 linhas. Uma célula por quadro, no máximo. - Ignora bola cujo `source` está em `{predicted, remembered, lost}`. - Sem `field_x`/`field_y` finitos: incrementa `frames_without_field_point` e não pinta. - Fora do campo, com tolerância de borda `BALL_HEATMAP_EDGE_TOLERANCE_CM = 100`: incrementa `frames_outside_field` e não pinta. A tolerância existe porque o jitter da homografia coloca a bola alguns centímetros além da linha. - Dentro da tolerância, a coordenada é cortada para `[0, comprimento]` e `[0, largura]`.

Coluna = `min(11, int(x / comprimento × 12))`. Linha = `min(7, int(y / largura × 8))`. - `observed_points` conta só as células pintadas. `max_cell_count` é o maior valor de uma célula.

No frontend, `cellIntensity` é a raiz quadrada de `célula / máximo`, para uma zona pouco usada continuar visível ao lado da mais usada. A cor vai de amarelo `(255, 235, 59)` a laranja `(255, 152, 0)` na metade inferior, e de laranja a vermelho `(229, 57, 53)` na metade superior. Alpha = `0,25 + 0,6 × intensidade`. O canvas desenha o campo (área, meia-lua, círculo central) por cima das células.

5. Implementação. Commit `bf0cc80` (19/10/2026), "Accumulate an observed-ball heatmap and draw it on the match panel". Este é o HEAD da branch. - `ball_heatmap.py` (novo): `BallHeatmapAccumulator`, `observe`, `_cell_for`, `as_dict`. - `run_yolo_analysis.py`: `ball_heatmap.observe` depois de atribuir coordenadas de campo e antes da posse; `ball_heatmap` no payload de progresso e no `result_json`. - `ballHeatmapProgress.js` (novo): valida e copia. - `YoloAnalysisProvider.js`, `AnalysisService.js`: `progress_stats.ball_heatmap`. - `public/js/ballHeatmap.js` (novo): `readBallHeatmap`, `cellIntensity`, `heatColor`, `hottestCell`, `drawBallHeatmap`. `readBallHeatmap` prefere `progress_stats` e cai para `result_json`. - `analysisViewer.js`, `app.html`, `style.css`: bloco "Mapa de calor da bola". - Testes: `scripts/test-ball-heatmap.py`, `scripts/test-ball-heatmap-panel.mjs`.

Não houve reprocessamento de vídeo. Cada quadro incrementa no máximo uma célula. Pass detector, lógica de posse, BallMemory, BallTracker, RF-DETR, TeamClassifier e o detector de classe não foram alterados.

6. Fluxo. Homografia do quadro ↓ detecção de bola com `field_x`/`field_y` e `source` ↓ `observe` ↓ se a fonte não é observada, retorna ↓ se o ponto é finito e cai na grade (com 100 cm de margem), incrementa a célula ↓ `as_dict` ↓ payload `ball_heatmap` ↓ `drawBallHeatmap`.

7. Problemas encontrados. Este ambiente não tem `torch` nem os pesos, então o pipeline não rodou aqui. Os mapas mostrados na sessão foram **sintéticos**, gerados a partir dos pontos de uma execução anterior, não de uma reanálise neste agente. Números relatados naquela execução do vídeo de 401 frames: - quadro 90: 77 bolas na grade, zona mais quente 40–50 m com 61% do máximo, 3 fora do mapa; - quadro 250: 214 bolas, 22%, 9 fora do mapa; - quadro 401: 341 bolas, célula mais quente com 13,8% do máximo, largura 17,5–26,3 m, 16 sem homografia ou fora do campo.

`hottestCell` devolve a primeira célula de maior contagem na ordem linha-coluna. Empate fica com a de menor linha e, na linha, a de menor coluna.

8. Por que essa solução. A grade 12 × 8 dá células de 10 m por 8,75 m, legíveis num cartão sem parecer um histograma de um quadro só. A raiz quadrada evita que a célula do lance parado apague o resto. Recusar bola prevista impede o mapa de desenhar a BallMemory.

9. Impacto. Worker (acumulador e campo), Node (cópia) e frontend (canvas). Nenhum consumidor antigo de `passes` ou `possession_by_team` mudou de formato.

10. Resultado. Mapa de calor da bola observada no painel, ao vivo e no resultado salvo. Testes que passaram nesta etapa: `test-ball-heatmap.py`, `test-ball-heatmap-panel.mjs`, `test-pass-panel.mjs`, `test-pass-detector.py`, `test-possession-stats.py`. Limitações que permanecem: bola sem homografia não pinta; a direção de ataque não é conhecida, então a grade não tem lado ofensivo; os buracos de bola dos quadros 245 e 270–276 (Seção 3.27) continuam sem ponto.

---

4. Estatísticas implementadas e como são calculadas

Esta seção descreve o cálculo **como está no código** de `cursor/yolov8-video-analysis-5e9e` no commit `bf0cc80`. Os números de exemplo vêm das execuções do vídeo de 401 frames relatadas nas seções 3.32 a 3.44. O painel não recalcula nada: ele lê o que o worker gravou.

4.1 Máquina de posse (não é um cartão, é a entrada dos outros)

`PossessionEngine` (`possession.py`) confirma um possuidor e publica o snapshot que o detector de passes e o acumulador de posse leem.

Constantes:

Nome	Valor	Papel
`POSSESSION_CONFIRM_FRAMES`	5	quadros seguidos para abrir a posse
`POSSESSION_LOSE_FRAMES`	8	quadros sem suporte para perder a posse
`POSSESSION_SWITCH_FRAMES`	8	quadros seguidos de outro jogador para trocar
`CONTROL_RADIUS_CM` / `CONTROL_RADIUS_PX`	250 cm / 90 px	raio para ganhar
`HOLD_RADIUS_CM` / `HOLD_RADIUS_PX`	400 cm / 140 px	raio em que o possuidor atual se mantém
`CONTEST_MARGIN_CM` / `CONTEST_MARGIN_PX`	70 cm / 25 px	margem em que o mais próximo não toma a posse
`TEAM_NAME_BY_ID`	`{0: "Team 0", 1: "Team 1"}`	nome gravado no snapshot

`build_ball_state` escolhe a bola do quadro: a observada, se existe; senão, se `internal_alive`, a prevista ou a lembrada; senão, nenhuma. Em `update`, só `source == "observed"` (`allow_new`) abre ou troca a posse. O possuidor atual permanece dentro do raio de manutenção. Um jogador mais próximo, dentro da margem de disputa, não toma a posse.

Métricas internas, não mostradas no cartão: `frames_with_possessor`, `confirmed_gains`, `losses`, `switches`.

4.2 Passes

O evento é a troca de possuidor confirmado do mesmo time. A trajetória da bola só altera o placar. A ausência de trajetória não veta o passe.

Quando vira passe, e quando não vira. `PassDetector.update` (`pass_detector.py`):

1. Cada quadro entra no `flight_log` (janela de 8 s). A bola só é guardada se `source` é `None` ou `"observed"`.
2. Sem possuidor, e existindo um último possuidor, abre um `pending`.
3. O mesmo possuidor, sem pending, apenas atualiza a memória.
4. O pending volta para o mesmo `track_id`: tenta a tabela (Seção 4.2.4). Se não houver parede, conta **condução** e não conta passe.
5. Outro possuidor: `_resolve`, nesta ordem: - `contested` (os dois times dentro do raio de disputa) → **disputa**, não é passe; - times conhecidos e diferentes → **troca de posse**, não é passe; - um dos times fora de `{"Team 0", "Team 1"}` → `discarded_unstable_possession`; - senão `_prepare_pass` e `_record_prepared`.

Distância. Primeiro A e B no mesmo quadro (`same_frame`), no quadro de soltura ou no de chegada. Se não houver os dois em coordenadas de campo, usa as posições guardadas em quadros diferentes (`cross_frame`). Sem coordenada de campo, a distância é `None` e o corte de curto/longo não se aplica; o placar também perde os 10 de distância plausível.

Tempo. Soltura = último quadro ≤ perda da posse em que A está a até 250 cm (ou 90 px) da bola observada. Chegada = primeiro quadro depois disso em que B está nesse raio. Se a chegada não for vista, usa-se o quadro em que B foi confirmado.

```
flight_seconds        = (arrival_frame - release_frame) / fps
reception_lag_seconds = (gained_frame - arrival_frame) / fps
gap_seconds           = (gained_frame - lost_frame) / fps
timing_seconds        = gap_seconds
se arrival foi observada e reception_lag_seconds ≤ 1,0:
    timing_seconds = min(gap_seconds, flight_seconds)
max_timing            = clamp(distance_cm / 800 + 0,5,  1,0,  4,0)
                      = 1,0 se não há distância
faixa                 = "base" se timing_seconds ≤ 1,0, senão "extended"
```

800 cm/s é `PASS_MIN_BALL_SPEED_CM_S`. 0,5 s é `PASS_RECEPTION_ALLOWANCE_SECONDS`.

Cortes, nesta ordem, antes do placar. O evento vai para `rejected_events` com `reason`:

Condição	`reason`	Contador
`timing_seconds > max_timing`	`timeout`	`discarded_timeout`
distância `< 150` cm	`short`	`discarded_short_passes`
distância `> 7000` cm	`long`	`discarded_long_passes`
velocidade implícita `> 4000` cm/s	`implausible_speed`	`discarded_implausible_speed`
adversário sozinho na bola por `max(3 quadros, 0,25 s)` entre soltura e chegada	`intercepted`	`discarded_intercepted`

A velocidade implícita só existe se as duas pontas foram observadas e há distância. Ela usa o intervalo mais longo: `distance_cm / ((gained_frame - release_frame) / fps)`. O campo `implied_speed_mps` é esse valor dividido por 100, com duas casas. (A coordenada de campo está em centímetros, então dividir por 100 produz m/s.)

Placar. `_pass_score_components`:

Componente	Pontos	Quando entra
`same_team`	40	sempre (o `_resolve` só chama o placar se os times são iguais e conhecidos)
`possessor_change`	30	sempre
`short_interval`	10 ou 0	10 só na faixa base
`plausible_distance`	10 ou 0	10 se a distância não é `None`
`trajectory`	+10, −20 ou 0	`compatible`, `incompatible`, ou `missing`
`unverified_extension`	−20 ou 0	−20 na faixa estendida, salvo se a trajetória é `compatible` **e** `in_flight_samples ≥ 2`

`in_flight_samples` conta quadros com bola observada estritamente entre soltura e chegada.

O passe entra em `events` e incrementa `total_passes` se a soma é ≥ `PASS_SCORE_THRESHOLD` (80). Abaixo disso vai para `unconfirmed_events` e incrementa `unconfirmed_candidates`. Não aparece no cartão.

Trajetória (`_trajectory_label`): série de distâncias bola–receptor dentro de cada quadro. A mediana do começo e a do fim (até 3 amostras de cada lado) decidem. Aproxima do receptor pelo menos `max(movimento mínimo, 0,5 × distância inicial)` → `compatible`. Afasta pelo menos o movimento mínimo (80 cm ou 20 px) → `incompatible`. Caso contrário `missing`. Sem duas amostras, cai na trajetória entre quadros (`_cross_frame_trajectory`), pelo produto escalar do deslocamento da bola com o vetor até o receptor.

O que o cartão soma. `countPassesByTeam` (`passCounts.js`) percorre `passes.events` e conta `event.team === "Team 0"` e `event.team === "Team 1"`. Um evento com outro `team`, ou sem `team`, não entra. Não há filtro extra por `confidence` nessa função: quem está em `events` já passou do limiar 80.

Contadores do perfil (`pass_detection` no `*.profile.json`), que o cartão não mostra: `total_passes`, `raw_candidate_passes`, `discarded_timeout`, `discarded_short_passes`, `discarded_long_passes`, `discarded_implausible_speed`, `discarded_intercepted`, `discarded_unstable_possession`, `extended_window_passes`, `possession_gap_passes`, `turnovers`, `disputes`, `conductions`, `wall_passes`, `unconfirmed_candidates`. `discarded_direction_check` existe no conjunto de contadores e o detector atual não o incrementa. O perfil não guarda a lista de eventos. As listas ficam em `events`, `unconfirmed_events`, `rejected_events`, `turnover_events` e `dispute_events`, no objeto de métricas e, para os passes contados, em `result_json.passes.events`.

No vídeo de 401 frames, depois da tabela (Seção 3.41), o cartão ficou em Team 0 = 0 e Team 1 = 7.

4.2.4 Tabela (dois passes num retorno)

Só roda quando o pending ia virar condução. `_find_wall`:

- time de A conhecido e pending não contestado;
- nenhum adversário sozinho na bola por `max(3, ceil(0,25 × fps))` quadros;
- o jogador mais próximo da bola observada, dentro de 120 cm (ou 45 px), é um companheiro diferente de A; se os dois times estão nesse raio no mesmo quadro, não há parede;
- a bola se aproxima desse companheiro, se afasta dele e volta em direção a A, cada trecho com pelo menos 80 cm (ou 20 px).

Cada lance passa pelo mesmo `_prepare_pass`. Se um deles tem status em `{timeout, short, long, implausible_speed, intercepted}`, nenhum dos dois é gravado e o retorno conta como condução. Lance com placar < 80 não está nesse conjunto. Cada lance contado incrementa `wall_passes` e, se o pending teve quadro sem bola, `possession_gap_passes`.

4.3 Posse por time

`PossessionTeamAccumulator.observe` (`possession_stats.py`) não decide posse. Soma o time do snapshot:

```
se team == "Team 0":  frames_team_0 += 1
se team == "Team 1":  frames_team_1 += 1
senão:                frames_without_possessor += 1
                      e, se o último time era conhecido, incrementa o bloco aberto
```

`fps` não positivo ou não numérico vira 30.

```
frames_total = frames_team_0 + frames_team_1 + frames_without_possessor
percent_*    = round(100 * frames_* / frames_total, 1)     # 0 se frames_total é 0
seconds_*    = round(frames_* / fps, 3)
```

Os três percentuais são sobre todos os quadros. Não são renormalizados entre si além do arredondamento de uma casa.

Reatribuição. O bloco aberto é só `{team, frames}` do último possuidor. No quadro em que volta a haver possuidor, `_take_new_pass_team` olha os eventos de `passes.events` ainda não vistos (`_seen_pass_events`). Se todos os novos do tipo `"pass"` têm o mesmo time, e esse time é o do bloco, `_reassign` move `min(bloco, frames_without_possessor)` quadros de "sem posse" para o time e soma o mesmo valor em `frames_reassigned_team_0` ou `frames_reassigned_team_1`. Esses dois contadores

são auditoria: não entram em `frames_total` nem no percentual. O quadro corrente já foi somado no time antes da transferência.

Não move o bloco: condução, troca de posse, disputa, passe não confirmado, eventos novos de times misturados, ou o vídeo terminar com o bloco aberto.

O frontend (`possessionCounts.js`) lê `progress_stats.possession_by_team` e, na falta, `result_json.possession_by_team`. `formatPossessionPercent` mostra uma casa. `formatPossessionSeconds` mostra `"x.x s"` abaixo de 60 s e `m:ss` a partir de 60 s.

No vídeo de 401 frames, a 30 fps, o final depois da tabela foi: Team 1 = 374 quadros, 93,3%, 12,5 s; sem possuidor = 27 quadros, 6,7%, 0,9 s; Team 0 = 0.

4.4 Trocas de posse

Não há uma segunda fórmula. O cartão mostra `passes.turnovers`, que é o inteiro `PassDetector.turnovers`.

Sobe em 1 quando `_resolve` encontra dois possuidores confirmados, os dois com time em `{"Team 0", "Team 1"}`, e os times são diferentes. A disputa retorna antes e não incrementa. A confiança gravada no evento, e **não** mostrada no cartão, é:

```
70 + 10 se a trajetória é compatible
70 - 10 se a trajetória é incompatible
70     se a trajetória é missing
```

`passProgressFromPayload` copia o número com `Math.round`. Valor ausente, não finito ou ≤ 0 vira 0. A confirmação continua em 5 quadros: uma troca de um quadro de detecção não conta.

4.5 Mapa de calor da bola

`BallHeatmapAccumulator` (`ball_heatmap.py`). Campo de 12000 cm × 7000 cm. Grade de 12 × 8. Uma visita por quadro.

```
ignora se não há bola, ou se source ∈ {predicted, remembered, lost}
sem field_x/field_y finitos → frames_without_field_point += 1
fora de [-100, comprimento+100] × [-100, largura+100] → frames_outside_field += 1
senão:
    x = clamp(field_x, 0, 12000)
    y = clamp(field_y, 0, 7000)
    coluna = min(11, int(x / 12000 * 12))
    linha  = min(7,  int(y / 7000 * 8))
    cells[linha][coluna] += 1
    observed_points += 1
```

Cada célula mede 1000 cm × 875 cm (10 m × 8,75 m).

No canvas (`ballHeatmap.js`):

```
intensidade = 0 se a célula ou o máximo é 0
intensidade = sqrt(min(1, célula / max_cell_count))
cor: amarelo → laranja se intensidade < 0,5; laranja → vermelho senão
alpha = 0,25 + 0,6 × intensidade
```

`hottestCell` percorre a grade em ordem de linha e coluna e fica com a maior contagem. A faixa em metros é `coluna × (comprimento/12) / 100` até a coluna seguinte, e o análogo para a linha.

`readBallHeatmap` recusa um objeto cuja matriz não tem `rows` linhas de `columns` células. Nesse caso o painel desenha a grade vazia de 12 × 8.

4.6 Travas que não são estatística e mudam os números

Time. `TeamTrackLock` (`roboflow_team.py`): trava depois de 3 classificações iguais (`TEAM_LOCK_CONFIRM_STREAK = 3`); 3 contradições seguidas destrava (`TEAM_LOCK_UNLOCK_STREAK = 3`). O refresh do classificador é só voto. A âncora pinta o agrupamento mais azul como time 0. Um `team_id` instável trocaria o `team` do passe e a barra de posse.

Classe. `PersonClassLock` (`person_class.py`): trava jogador, goleiro ou árbitro depois de 3 quadros iguais e destrava com 3 contradições. Rejeita não-pessoa por aspecto `< 0,16` (fino), `> 1,8` (largo) ou aspecto `< 0,24` com matiz de bandeira `> 45%` no terço superior. A suavização antiga de 9 quadros (`stabilize_player_referee_class`) continua no código. Um árbitro classificado como jogador entra na posse.

Goleiro. `resolve_goalkeepers_team_id` atribui o time do centróide de pés mais próximo. O goleiro pode ser possuidor.

---

5. Funcionalidades pendentes e limitações conhecidas

Só entra aqui o que foi discutido e **não** implementado, ou o que foi medido e ficou em aberto.

1. **Posse por faixa do campo.** Faixas 0–40 m, 40–80 m e 80–120 m, com rótulos esquerda, meio e direita. Discutida no turno 78. Não rotular ataque e defesa: a direção de ataque não é conhecida e inverte no intervalo.
2. **Duração média da posse.** Terceira da ordem do turno 78. Não implementada.
3. **Mapa de calor de jogadores.** Não recomendado enquanto 22 trilhas, goleiro e árbitro se misturam na mesma grade.
4. **Distância e velocidade por jogador.** A homografia salta entre quadros e a trilha quebra. Falta um teto de salto e de velocidade. A velocidade máxima é o caso que mais infla.
5. **Cartão de recuperações.** Redundante com a troca de posse já exibida.
6. **Chutes, gols e rede de passes.** Não há classe de chute. O chute final do vídeo de teste (trilha 74, quadros 392–401) foi excluído por não ter receptor.
7. **Passe longo com a bola oculta.** Proposta do turno 47, não feita: retirar o −20 da faixa estendida quando as duas pontas são observadas, a trajetória é compatível e a velocidade fica entre 8 e 40 m/s. A BallMemory de 5 quadros continua curta para preencher o meio do voo. Simulação a 30 fps, passe de 30 m: 1,20 s de oclusão dá placar 100; 1,33 s, 2 s e 3 s dão 70; 5 s dá timeout.
8. **Merge com `main` (`7f4dd14`).** Não commitado. Dois conflitos permanecem: nomes `"Team 0"`/`"Team 1"` contra `"blue"`/`"red"`, e a política da bola (`keep_recoverable_ball_candidates`, confiança 0,02, fallback e `weak_yolo_continuation`) contra a lista crua com limiar 0,25. Apagar a atribuição de `weak_yolo_continuation` e manter a leitura gera `NameError`. O aviso do GitHub continua porque o remoto não tem esse merge.
9. **Divergência de 2 passes na tela contra 5 nesta sessão.** O script de leitura do SQLite foi entregue e a saída não voltou. Sem o `result_json` da análise que mostrou 2, não há como decidir se o banco tem 2 (versão antiga) ou 5 (defeito de leitura). Não há teste que fixe "este vídeo tem 5 passes".
10. **Desempenho.** O perfil do clipe de 150 frames (turno 22) mediu cerca de 179 s para 6,1 s de vídeo: jogadores 26,1%, campo 26,3%, bola 25,3%, ajuste de times 7,9%, predição 5,6%. Rodar o modelo de campo a cada N quadros, usar GPU e mudar `imgsz` foram medidos como hipóteses e não implementados.
11. **Buracos de bola que continuam.** No vídeo da bola laranja, o quadro 245 tem confiança 0,21, abaixo de 0,25, e o trecho 270–276 segue sem caixa utilizável. A estratégia de tiles e de `imgsz` maior foi medida no turno 33 e não entrou no código.
12. **RF-DETR.** O backend existe (`BALL_DETECTOR_BACKEND_DEFAULT = "yolo"`). A busca preditiva e a híbrida só rodam com backend `rfdetr`. Os pesos `weights/checkpoint_best_regular.pth` não estão neste ambiente, então o RF-DETR não foi reexecutado aqui. O `InferenceSlicer` está desligado de propósito (`ball_slicer = None`); o callback permanece.
13. **Precisão dos 89 quadros do fallback.** No turno 35 a gravação já não estava na máquina e o perfil só tinha agregados. Não há como dizer quantos desses 89 eram bola de verdade.
14. **Reidentificação espacial de trilhas.** Proposta no turno 17, junto com a análise do Simo-03 e o `BallEventLayer` (oito regras anti-falso-passe). Não implementada. O ByteTrack continua a única identidade.
15. **Commits de passe por intervalo de posse.** `2732558` e `e161b65` (PR #17) foram excluídos de propósito no turno 40, quando a integração trouxe o detector por placar. Não estão na branch.

16. **Ambiente desta sessão.** Sem `torch`, sem `ultralytics` e sem os pesos, o worker não processa vídeo aqui. Os mapas de calor mostrados na sessão do heatmap foram sintéticos.

---

6. Apêndice A — Linha do tempo de commits e PRs

Commits da branch `cursor/yolov8-video-analysis-5e9e`, do mais recente ao mais antigo. A data é a do `git log`. Quando o texto do PR #12 é a única fonte de um detalhe de julho ou agosto, o relatório não o usa: o corpo desse PR foi sobrescrito com o texto de posse, e os detalhes daqueles meses saíram do código e da mensagem do commit.

Commit	Data	O que entrou	Seção
`bf0cc80`	2026-10-19	Mapa de calor da bola observada	3.44
`14fb7a5`	2026-10-19	Trocas de posse no painel	3.42
`0b217e0`	2026-10-18	Tabela como dois passes	3.41
`78940e3`	2026-10-18	Reatribuição do intervalo de posse	3.39
`aae84a1`	2026-10-18	Posse por time	3.38
`3d97765`	2026-10-12	Passes do backend no painel	3.34
`78886a4`	2026-10-12	Tempo de voo e orçamento por distância	3.32
`e88f3ec`	2026-09-22	PersonClassLock	3.24
`effbbde`	2026-09-22	Nomes Team 0 e Team 1	3.23
`b47da34`	2026-10-04	Passe por placar de posse	3.29
`d06e77a`	2026-09-27	Fallback da classe `ball` do modelo de jogadores	3.28
`a503632`	2026-09-27	Bola fraca próxima preferida à forte distante	3.26
`78a24f2`	2026-09-27	Bola fraca só continua a trilha	3.26
`1fb0cef`	2026-09-22	Âncora de cor e lock de time	3.22
`c7c8c6f`	2026-09-21	PassDetector v1, PR #13	3.20
`69e1d19`	2026-09-20	Merge de `origin/main`	3.19
`da50b21`	2026-09-20	PossessionEngine	3.19
`3674ed2`	2026-09-19	Painel da partida e contrato de eventos	3.18
`d98e913`	2026-09-10	Próximos passos do deck do Seminário III	3.16
`0a53870`	2026-09-10	Deck só de texto do Seminário III	3.16
`169fcb7`	2026-09-10	Deck do Seminário III	3.16
`c67eace`	2026-08-25	Validação física da bola	3.15
`f71f1dc`	2026-08-16	Recuperação semi-global antecipada	3.15
`5cfb5c4`	2026-08-16	Etapa semi-global da busca híbrida	3.15
`804682c`	2026-08-15	Tiling global e smart crop	3.15
`09e2bac`	2026-08-14	ROI preditivo para RF-DETR	3.15
`26671f4`	2026-08-12	Carga do checkpoint RF-DETR	3.15
`7960aaf`	2026-08-12	Backend RF-DETR SoccerNet	3.15
`399b73d`	2026-08-12	Diagnóstico por quadro da bola	3.14
`4cecdc6`	2026-08-11	Memória, predição e fusão por ROI, PR #10	3.14
`12e6e73`	2026-08-12	Merge de `origin/main`	3.14
`5a11878`	2026-08-12	Contagem da continuidade pela memória interna	3.14
`80128b3`	2026-08-12	Memória da bola e fusão por ROI	3.14
`4db3f11`	2026-08-09	Remoção do overlay da máscara de campo	3.13
`d761c60`	2026-08-08	Remoção do filtro de pessoas pela máscara	3.13
`e4cfccb`	2026-08-08	Cache do TeamClassifier por trilha	3.12
`96c87c5`	2026-08-03	Perfil do pipeline	3.13

Commit	Data	O que entrou	Seção
`30a400b`	2026-08-03	Homografia Roboflow	3.13
`97700b5`	2026-08-03	ByteTrack e suavização jogador/árbitro	3.11
`4bd2549`	2026-08-02	Time do goleiro pelo centróide	3.12
`f7aec46`	2026-08-02	Ajuste limitado do TeamClassifier	3.12
`e978a5b`	2026-08-01	Limiares de amostra do TeamClassifier	3.12
`1abca2f`	2026-08-01	Qualidade do recorte de treino	3.12
`a329aaa`	2026-08-01	TeamClassifier no pipeline	3.12
`72f8ba8`	2026-08-01	ByteTrack	3.11
`2c92ba9`	2026-08-01	InferenceSlicer desligado	3.10
`aafb038`	2026-07-31	Detector de jogadores Roboflow, PR #11	3.11
`8d12676`	2026-07-29	Argumentos do slicer no supervision 0.29.1	3.10
`0bd7d31`	2026-07-29	Fluxo do slicer Roboflow	3.10
`c794580`	2026-07-28	BallTracker	3.10
`401dcd4`	2026-07-28	Fallback da bola desligado	3.10
`e6111f2`	2026-07-27	Modelo dedicado de bola	3.10
`95e4c7e`	2026-07-27	Merge da branch de keypoints	3.9
`e6d9ba4`	2026-07-27	Modelo de keypoints do campo, PR #9	3.9
`81792a2`	2026-07-27	Script de download do modelo de campo	3.9
`a4c6150`	2026-07-06	Só as linhas externas no polígono	3.8
`4122de8`	2026-07-06	Polígono cortado pelas linhas brancas	3.8
`10c5490`	2026-07-06	Linhas brancas no filtro de jogadores	3.8
`ca626aa`	2026-07-05	Frame de debug em memória	3.7
`ad57900`	2026-07-04	Merge do PR #8	3.6
`9052472`	2026-07-05	Botão de seleção de arquivos vermelho	3.6
`849278a`	2026-07-05	O mesmo botão, no PR #8	3.6
`64502d6`	2026-07-04	Log temporário do frame ao vivo, PR #7	3.6
`98f1727`	2026-06-30	Merge do PR #6	3.6
`7d2f6f6`	2026-06-29	Constante do frame ao vivo	3.6
`f2acd3a`	2026-06-29	Frame de debug ligado por padrão, PR #6	3.6
`dc789cc`	2026-06-27	Atualização de `run_yolo_analysis.py`	—
`29eedd5`	2026-06-10	Merge do WinError 5	3.6
`bf31cee`	2026-06-10	Merge mantendo filtro de campo e retry	3.6
`0e2f387`	2026-06-10	Retry de permissão no frame ao vivo, PR #5	3.6
`29bd930`	2026-06-10	O mesmo retry, antes do merge	3.6
`965de4a`	2026-06-10	Máscara de campo no PR #4	3.5
`2baefe8`	2026-06-10	Merge de `origin/main`	3.5
`70dff1d`	2026-06-08	Máscara HSV de campo	3.5
`7b54e66`	2026-06-08	YOLOv8 no PR #2	3.2
`ab311ae`	2026-06-08	Frame de debug e métrica de IoU	3.4
`8262ac9`	2026-06-06	Resolvedor de Python no Windows, PR #3	3.3
`645a56f`	2026-06-05	Saída H.264 para o navegador	3.2
`346e860`	2026-06-05	Visualização da análise YOLOv8	3.2
`6c8c3b4`	2026-05-30	Carga inicial do repositório	3.1

Pull requests citados no desenvolvimento:

PR	Assunto	Commit de referência
#2	Análise YOLOv8	`7b54e66`
#3	Python no Windows, ignorando o atalho da Microsoft Store	`8262ac9`
#4	Filtro de jogadores pela máscara de campo	`965de4a`
#5	Retry do WinError 5	`0e2f387`
#6	Frame ao vivo por padrão	`98f1727`
#7	Log temporário do frame ao vivo	`64502d6`
#8	Botão vermelho de seleção	`ad57900`
#9	Keypoints do campo	`e6d9ba4`
#10	Memória, predição e ROI	`4cecdc6`
#11	Detector de jogadores Roboflow	`aafb038`
#12	Corpo sobrescrito com o texto de posse; não usar como fonte	—
#13	PassDetector v1	`c7c8c6f`
#17	Passes por intervalo de posse (`2732558`, `e161b65`); excluído no turno 40	—
#19	Fallback da classe `ball`	`d06e77a`
#20	Passe por placar	`b47da34`
#21	Esta branch, contra `main`	`bf0cc80`

Cherry-picks da integração (turno 40), na ordem em que entraram na branch oficial: `1fb0cef` (era `945ba0d`), `78a24f2` (era `0a21239`), `a503632` (era `4f81770`), `d06e77a` (era `b3bd8e0`), `b47da34` (era `ea7408a`), `effbbde` (era `9a29e53`), `e88f3ec` (era `646c698`).

7. Apêndice B — Testes automatizados

Os scripts ficam em `conversor-video/scripts/`. Eles cobrem a regra isolada. Nenhum deles reprocessa um vídeo com os pesos, porque este ambiente não tem `torch` nem os checkpoints.

Script	O que trava
`test-python-resolver.js`	candidatos `py -3`, `py`, `python`, `python3`; caminhos `WindowsApps` ignorados; `YOLO_PYTHON_BIN` tem prioridade
`test-yolo-spawn-error.js`	erro de spawn do worker chega ao provider
`test-team-classifier-stability.py`	âncora azul → time 0, lock em 3, destrave em 3, rejeição de recorte
`test-person-class-lock.py`	lock de classe, bandeirinha, caixa fina e caixa larga
`test-ball-recall-filter.py`	bola fraca só continua a trilha; fraca próxima ganha da forte distante
`test-player-ball-fallback.py`	fallback só com trilha aberta e detector dedicado vazio; caixa ≤ 80 px e confiança ≥ 0,25
`test-possession-engine.py`	confirmação em 5, perda em 8, troca em 8, raios de campo e de pixel, nomes Team 0 e Team 1
`test-possession-stats.py`	percentual sobre todos os quadros, bloco aberto, reatribuição só com passe novo do mesmo time
`test-possession-panel.mjs`	leitura de `progress_stats` e de `result_json`, formatação
`test-pass-detector.py`	placar, faixas, timeout, velocidade, interceptação, condução, tabela (6 casos), fixtures a 10 fps
`test-pass-panel.mjs`	cartão começa em zero, sobe com o payload, disputa não conta como troca, valor final permanece
`test-ball-heatmap.py`	bola prevista não pinta, tolerância de 100 cm, uma célula por quadro
`test-ball-heatmap-panel.mjs`	normalização da grade, intensidade em raiz quadrada, cor
`download-pitch-model.js`	download do modelo de keypoints; não é um teste de regra

Na etapa do mapa de calor, passaram juntos: `test-ball-heatmap.py`, `test-ball-heatmap-panel.mjs`, `test-pass-panel.mjs`, `test-pass-detector.py` e `test-possession-stats.py`.

8. Apêndice C — Vídeos de teste usados

Dois vídeos aparecem nas medições. Os números abaixo são os das execuções relatadas em cada seção. Uma reexecução pode divergir (Seção 3.36).

Clipe de 150 frames. Trecho de 6,1 s, 1920×1080, Barcelona × Osasuna. Arquivo citado como `5b29ff45-…mp4` e, numa execução posterior, `5c06736d-9f1d-4c67-a213-d24e1872fffb.mp4`. Foi o vídeo padrão até o trabalho de recall da bola. Na integração (Seção 3.30), com `--ball-detector yolo`: 150 quadros, 150 bolas, 2072 jogadores, posse em 144 quadros, homografia válida nos 150, times 888 e 1184, 14 trilhas travadas, 0 passes. A posse da trilha 10 confirmou no quadro 7. O perfil do turno 22 mediu cerca de 179 s de processamento para 6,1 s de vídeo.

Gravação de tela "Gravação de Tela 2026-09-21 180218". 401 quadros, 30 fps, 1338×750, bola laranja. Usada para passes, posse e estatísticas a partir da Seção 3.27. Resultados que ficaram registrados, na versão final do detector (depois de `0b217e0`):

- 7 passes, todos Team 1. Dois deles são os lances da tabela que fecha no quadro 164.
- Trocas de posse: 0. Todos os possuidores confirmados são Team 1.
- Posse: Team 1 em 374 quadros (93,3%, 12,5 s); sem possuidor em 27 quadros (6,7%, 0,9 s), que são o início do vídeo e o chute final.
- Mapa de calor, na execução sintética da Seção 3.44: 341 pontos na grade ao final, 16 sem homografia ou fora do campo.

Antes da tabela a mesma gravação tinha 5 passes e 76,3% de posse do Team 1 (Seção 3.39). Antes do voo tinha 3 passes e dois descartes por timeout (Seção 3.31). Essas três contagens são a mesma gravação em três versões do detector, não três vídeos.

Fim do relatório. Branch `cursor/yolov8-video-analysis-5e9e`, commit `bf0cc80`, 19/10/2026.